# Intelligenza Artificiale

UniVR - Dipartimento di Informatica

**Fabio Irimie, Legion**

1° Semestre 2025/2026

# Indice


**1 Introduzione** **5**
- 1.1 Tipi di intelligenza artificiale . . . 5
  - 1.1.1 Autonomous agents . . . 5
  - 1.1.2 Data analysis . . . 5
  - 1.1.3 Machine Learning . . . 5
  - 1.1.4 Time series analysis . . . 6
  - 1.1.5 Intelligent Agents . . . 6
- 1.2 Markov Decision Process (MDP) . . . 6
- 1.3 Generative AI . . . 7

**2 Agenti e ambiente** **7**
- 2.1 Razionalità . . . 8
- 2.2 PEAS . . . 9
- 2.3 Tipi di ambienti . . . 10
- 2.4 Agenti di problem solving . . . 10

**3 Ricerca nello spazio degli stati** **12**
- 3.1 Ricerca generale . . . 12
  - 3.1.1 Tree search . . . 12
  - 3.1.2 Stato e nodo . . . 12
  - 3.1.3 Tree search generale . . . 12
  - 3.1.4 Stati ripetuti . . . 13
- 3.2 Ricerca non informata . . . 13
  - 3.2.1 Breadth-first search . . . 14
  - 3.2.2 Uniform-cost search . . . 14
  - 3.2.3 Depth-first search . . . 15
  - 3.2.4 Iterative deepening search . . . 15
- 3.3 Ricerca informata . . . 17
  - 3.3.1 Best-first search . . . 17
  - 3.3.2 Greedy best-first search . . . 17
  - 3.3.3 A* search . . . 18
  - 3.3.4 Consistenza e ammissibilità . . . 18
  - 3.3.5 Euristiche . . . 19
- 3.4 Ricerca locale . . . 20
  - 3.4.1 Hill climbing . . . 21
  - 3.4.2 Simulated annealing . . . 22
  - 3.4.3 Local beam search . . . 23
  - 3.4.4 Algoritmi genetici . . . 23
- 3.5 Ricerca locale in uno spazio continuo . . . 24
  - 3.5.1 Gradient ascent/descent . . . 24
  - 3.5.2 Algoritmo di Newton-Raphson . . . 25
  - 3.5.3 Calcolo degli zeri del gradiente . . . 26
  - 3.5.4 Gradiente empirico . . . 26
- 3.6 Constrained satisfaction problem . . . 27
  - 3.6.1 Grafo dei vincoli . . . 28
  - 3.6.2 Problemi combinatori . . . 30
  - 3.6.3 Backtracking search . . . 31
  - 3.6.4 Inferenza . . . 32

3.6.5 Look Ahead . . . 33
3.6.6 Forward checking look ahead . . . 33
3.6.7 Arc consistency look ahead . . . 34
3.6.8 Forzare l'arc consistency . . . 35
3.6.9 Tree decomposition . . . 36

**4 Logical Agents 36**
4.1 Knowledge based agents . . . 36
4.2 Logica in generale . . . 38
4.2.1 Entailment (Derivazione logica) . . . 39
4.2.2 Inferenza . . . 39
4.2.3 Inferenza mediante enumerazione . . . 40
4.2.4 Metodi di dimostrazione . . . 40
4.2.5 Equivalenza logica . . . 41
4.2.6 Validità e soddisfacibilità . . . 41
4.3 Sistema di inferenza . . . 42
4.3.1 Proprietà di un sistema di inferenza . . . 42
4.4 Problema di deduzione . . . 43
4.4.1 Resolution . . . 43
4.4.2 Conversione in CNF . . . 43
4.5 Forward e backward chaining . . . 44
4.5.1 Forward chaining . . . 45
4.5.2 Backward chaining . . . 45
4.5.3 Differenze tra forward e backward chaining . . . 46

**5 Rappresentare l'incertezza 46**
5.1 Probabilità . . . 46
5.1.1 Decidere con incertezza . . . 47
5.1.2 Basi di probabilità . . . 47
5.1.3 Variabili casuali . . . 48
5.1.4 Eventi atomici . . . 48
5.1.5 Probabilità a priori . . . 48
5.1.6 Probabilità congiunta . . . 48
5.1.7 Probabilità continua . . . 49
5.1.8 Probabilità condizionata . . . 49
5.1.9 Inferenza per enumerazione . . . 50
5.1.10 Normalizzazione . . . 51
5.2 Indipendenza . . . 52
5.2.1 Indipendenza condizionata . . . 52
5.2.2 Regola di Bayes . . . 53

**6 Sequential Decision Making 53**
6.1 Decision trees . . . 55
6.1.1 Risoluzione di alberi di decisione . . . 56
6.2 Markov Decision Processes . . . 57
6.2.1 Proprietà . . . 57
6.2.2 Tipi di policy . . . 58
6.2.3 Decisione tra policy . . . 59
6.2.4 Valore di una policy . . . 59
6.2.5 Risolvere un MDP . . . 60

6.2.6 Equazioni di Bellman . . . 60
6.2.7 Value iteration . . . 61
6.2.8 Policy iteration . . . 62
6.3 MDP parzialmente osservabili . . . 63

**7 Machine learning** **64**
7.1 Concetti di base e terminologia . . . 64
7.2 Supervised learning . . . 64
7.3 Unsupervised learning . . . 65
7.4 Reinforcement learning . . . 66
7.5 Predizione . . . 66
7.5.1 Accuratezza della predizione . . . 67
7.5.2 Inferenza . . . 67
7.6 Stima di una funzione . . . 68
7.6.1 Metodi parametrici . . . 68
7.6.2 Metodi non parametrici . . . 69
7.7 Accuratezza del modello . . . 70
7.7.1 Interpretabilità e flessibilità . . . 70
7.7.2 Capire l'accuratezza . . . 71
7.7.3 Bias-variance tradeoff . . . 72
7.7.4 Stimare il test error (cross-validation) . . . 73
7.7.5 Leave one out cross validation . . . 73
7.7.6 K-fold cross validation . . . 74
7.7.7 Visualizzazione intuitiva del bias-variance tradeoff . . . 76
7.7.8 Confronto tra cross validation . . . 76
7.8 Regressione lineare . . . 77
7.8.1 Regressione lineare semplice (univariata) . . . 77
7.8.2 Trovare i pesi ottimali . . . 78
7.8.3 Gradient descent . . . 79
7.8.4 Stochastic gradient descent . . . 80
7.9 Reti neurali . . . 80
7.9.1 Feed-forward neural network . . . 80
7.9.2 Caolcolo di una rete neurale . . . 81
7.9.3 Funzione di attivazione . . . 83
7.9.4 Proprietà universale di approssimazione . . . 85
7.9.5 Addestramento di una rete neurale . . . 85
7.9.6 Back-propagation . . . 85
7.9.7 Deep neural networks . . . 86
7.10 Generative AI . . . 87
7.10.1 Rappresentare i concetti tramite embedding . . . 88
7.10.2 Attenzione . . . 88
7.10.3 Language models . . . 89
7.11 Reinforcement learning . . . 90
7.11.1 Conoscenza del modello . . . 91
7.11.2 Metodi model-based . . . 92
7.11.3 Metodi model-free . . . 92
7.11.4 Sample-based policy evalueation . . . 92
7.11.5 Temporal Difference Learning . . . 92
7.11.6 Q-learning . . . 93
7.11.7 Sample based Q-learning . . . 93

7.11.8 SARSA: alternativa on-policy al Q-learning . . . . . 94
7.12 Esplorazione vs Sfruttamento . . . . . 94
7.12.1 Funzioni di esplorazione . . . . . 95
7.13 Deep reinforcement learning . . . . . 96
7.13.1 Gradient Q-Learning . . . . . 96
7.14 Mitigare la divergenza . . . . . 97
7.14.1 Experience replay . . . . . 97
7.14.2 Target network . . . . . 97
7.14.3 Deep Q Network . . . . . 97

# 1 Introduzione

Nel 1950 Alan Turing pubblica un articolo intitolato "Computing Machinery and Intelligence" in cui propone un esperimento per determinare se una macchina può essere considerata intelligente. L'esperimento, noto come "test di Turing", coinvolge un interrogatore umano che comunica con due entità nascoste: una macchina e un essere umano. L'interrogatore deve fare domande a entrambe le entità e, basandosi sulle risposte, deve determinare quale delle due è la macchina. Se l'interrogatore non riesce a distinguere tra le risposte della macchina e quelle dell'essere umano, la macchina è considerata intelligente.

In futuro l'attenzione si è spostata sulla ricerca di metodi per risolvere problemi che richiedono intelligenza umana, utilizzando algoritmi e modelli matematici fino ad arrivare alle reti neurali e intelligenza artificiale.

**Definizione 1.1.** L'intelligenza artificiale è una disciplina che studia come **simulare** l'intelligenza umana in scenari complessi

## 1.1 Tipi di intelligenza artificiale

### 1.1.1 Autonomous agents

Sono sistemi che percepiscono l'ambiente e agiscono in modo autonomo per raggiungere obiettivi specifici.

### 1.1.2 Data analysis

Utilizzo di algoritmi per analizzare grandi quantità di dati e estrarre informazioni utili e correlazioni complesse.

### 1.1.3 Machine Learning

È lo sviluppo di algoritmi che permettono a dei modelli di apprendere dai dati di esempio e migliorare le loro prestazioni nel tempo senza essere esplicitamente programmati. Ad esempio riconoscimento di immagini.

L'apprendimento automatico è diviso in tre categorie principali:

- **Unsupervised learning**: il modello viene addestrato su un insieme di dati non etichettati, dove l'obiettivo è scoprire strutture nascoste o pattern nei dati senza avere risposte corrette predefinite.
- **Supervised learning**: il modello viene addestrato su un insieme di dati etichettati, dove ogni esempio di input è associato a una risposta corretta. L'obiettivo è che il modello impari a mappare gli input alle risposte corrette.
- **Reinforced learning**: il modello impara attraverso interazioni con l'ambiente, ricevendo ricompense o penalità in base alle azioni intraprese. L'obiettivo è massimizzare la ricompensa totale nel tempo.

### 1.1.4 Time series analysis

L'analisi delle serie temporali è un'area dell'apprendimento automatico che si concentra sull'analisi di dati collezionati nel tempo. Le serie temporali sono sequenze di dati misurati a intervalli regolari, come temperatura giornaliera, prezzi delle azioni o dati di vendita mensili. L'obiettivo dell'analisi delle serie temporali è identificare pattern, tendenze e stagionalità nei dati per fare previsioni future.

Gli approcci comuni per l'analisi delle serie temporali includono:

- **Riconoscimento di anomalie e cause**: è un processo di identificazione di dati o eventi che si discostano significativamente dal comportamento normale o atteso. Queste anomalie possono indicare problemi, errori o situazioni insolite che richiedono attenzione.
- **Generative transformers**: sono una classe di modelli che permettono di predirre il prossimo elemento in una sequenza di dati partendo dagli elementi precedenti, come ad esempio la parola successiva in una frase o il pixel successivo in un'immagine. Si sfrutta il concetto di **attenzione** per pesare l'importanza relativa delle diverse parti della sequenza di input durante la generazione dell'output.

### 1.1.5 Intelligent Agents

Un agente intelligente è un sistema che percepisce l'ambiente circostante attraverso sensori e agisce su l'ambiente per raggiungere un obiettivo specifico. Gli elementi chiave di un agente intelligente includono:

- **Performance measure**: misura il successo dell'agente nel raggiungere i suoi obiettivi
- **Rationality**: l'agente deve agire in modo da massimizzare la sua performance measure attesa

## 1.2 Markov Decision Process (MDP)

Un MDP è un modello matematico utilizzato per rappresentare problemi di decisione sequenziali. Gli elementi principali sono:

- **State**: rappresenta l'ambiente in un dato momento
- **Actions**: insieme delle azioni che l'agente può intraprendere
- **Transition model**: effetto che le azioni hanno sull'ambiente (potrebbero essere parzialmente incognite

$$\mathrm{T} : (state, \mathrm{action}) \rightarrow next_state$$

- **Reward**: valore **immediato** dell'esecuzione di un'azione

$$\mathrm{R} : (state, \mathrm{action}, next_state) \rightarrow real_number$$

- **Policy**: strategia che l'agente utilizza per decidere quale azione intraprendere in ogni stato con l'obiettivo di massimizzare la ricompensa totale attesa nel tempo

$$\pi : (state) \rightarrow \mathrm{action}$$

## 1.3 Generative AI

L'intelligenza artificiale generativa si riferisce a una classe di modelli di intelligenza artificiale che sono in grado di generare nuovi contenuti, come testo, immagini, musica o video, a partire da dati di addestramento. Questi modelli hanno miliardi di parametri e sono **preaddestrati** su grandi quantità di dati. In sostanza questi modelli "predicono il futuro" basandosi sui dati su cui sono stati addestrati e un **propmpt** (input dell'utente).

# 2 Agenti e ambiente

Gli agenti includono umani, robot, softbot, termostati ecc... La funzione dell'agente mappa lo storico di percezioni in azioni:

$$f : \mathcal{P}^* \mapsto \mathcal{A}$$

Il programma dell'agente è eseguito su architettura fisica per produrre la funzione f.

**Esempio 2.1.** Un esempio potrebbe essere un insieme di stanze $\{A, B\}$ e un robot aspirapolvere che può percepire la sua posizione e il contenuto della stanza. L'agente potrebbe quindi percepire $[A, Sporco]$ se ci fosse dello sporco nella stanza A. Le azioni potrebbero essere di movimento o pulizia. Tutto questo dipende dalla squenza di percezioni, ad esempio in una tabella:

| Percezione | Azione |
|---|---|
| $[A, Pulito]$ | Vai a B |
| $[A, Sporco]$ | Pulisci |
| $[B, Pulito]$ | Vai ad A |
| $[B, Sporco]$ | Pulisci |
| $[A, Pulito], [A, Pulito]$ | Vai a B |
| $[A, Pulito], [A, Sporco]$ | Pulisci |

Tabella 1: Esempio di tabella di percezioni e azioni

Non possiamo dire se questa è una funzione corretta perchè non abbiamo una **performance measure** che ci dica se l'agente sta facendo un buon lavoro.

**Definizione 2.1.** Se un agente ha $|\mathcal{P}|$ possibili percezioni, allora al tempo T avrà:

$$\sum_{t=1}^{T} |\mathcal{P}|^t$$

Se lo storico di percezioni è irrilevante, cioè se ad ogni percezione è associata un'azione la funzione viene chiamata **Reflex**.

## 2.1 Razionalità

Per definire l'intelligenza di un agente si utilizza una misura di performance che valuta la sequenza di percezioni.

**Esempio 2.2.** Tornando all'esempio del robot aspirapolvere si potrebbero assegnare i seguenti punteggi:

- Un punto per ogni stanza pulita per ogni unità di tempo
- Meno un punto per ogni mossa
- Penalizzazione per ogni stanza sporca

**Esempio 2.3.** Un altro esempio è il seguente ambiente:

- Ci sono 3 stanze $(A, B, C)$ e due robot $(r_1, r_2)$
- $r_1$ può sorvegliare solo $A$ e $B$ e $r_2$ solo $B$ e $C$
- $r_1$ inizia dalla stanza $A$ e $r_2$ dalla $C$
- Il tempo di percorrenza tra le stanze è 0
- Performance measure: minimizza il tempo in cui una stanza non è sorvegliata, cioè il tempo totale in cui una stanza non è visitata da nessun robot

Un possibile comportamento razionale potrebbe essere il seguente (alternata):

| Stato | A | B | C | Tempo |
|---|---|---|---|---|
| [A, C] | 0 | 1 | 0 | 1 |
| [B, C] | 1 | 0 | 0 | 2 |
| [A, C] | 0 | 1 | 0 | 3 |
| [A, B] | 0 | 0 | 1 | 4 |
| Average idleness | $\frac{1}{4}$ | $\frac{1}{2}$ | $\frac{1}{4}$ | Tot: $\frac{1}{3}$ |

Un altro comportamento potrebbe essere (fissata):

| Stato | A | B | C | Tempo |
|---|---|---|---|---|
| [A, C] | 0 | 1 | 0 | 1 |
| [B, C] | 1 | 0 | 0 | 2 |
| [A, C] | 0 | 1 | 0 | 3 |
| [B, C] | 1 | 0 | 0 | 4 |
| Average idleness | $\frac{1}{2}$ | $\frac{1}{2}$ | 0 | Tot: $\frac{1}{3}$ |

Entrambi i comportamenti hanno la stessa performance measure, ma il primo è migliore del secondo perchè penalizza meno una singola stanza rispetto alle

altre. Per capirlo bisogna non solo minimizzare la performance measure, ma anche minimizzare la varianza.

## 2.2 PEAS

Per progettare un agente intelligente bisogna definire l'ambiente in cui opera:

- **Performance measure**: come viene valutato il successo dell'agente
- **Environment**: il contesto in cui l'agente opera
- **Actuators**: i mezzi attraverso cui l'agente agisce sull'ambiente
- **Sensors**: i mezzi attraverso cui l'agente percepisce l'ambiente

**Esempio 2.4.** Prendiamo ad esempio un taxi automatico, il PEAS potrebbe essere:

- Performance measure:
  - Soddisfazione del cliente
  - Sicurezza
  - Efficienza del carburante
  - Rispetto delle leggi stradali
- Environment:
  - Traffico stradale
  - Condizioni meteorologiche
  - Segnali stradali
  - Pedoni e altri veicoli
- Actuators:
  - Volante
  - Acceleratore
  - Freni
  - Indicatori di direzione
- Sensors:
  - Telecamere
  - Lidar
  - Radar
  - Sensori di velocità
  - GPS

## 2.3 Tipi di ambienti

Gli ambienti possono essere classificati in base a diverse caratteristiche:

- **Osservabile**: se l'agente può percepire completamente lo stato dell'ambiente in ogni momento
- **Deterministico**: se l'azione dell'agente determina in modo univoco il prossimo stato dell'ambiente
- **Episodico**: se l'esperienza dell'agente è divisa in episodi indipendenti, cioè l'azione in un episodio non influisce sugli episodi successivi
- **Statico**: se l'ambiente non cambia mentre l'agente sta prendendo una decisione
- **Discreto**: se l'insieme di stati, azioni e percezioni è finito o numerabile
- **Singolo agente**: se l'agente opera da solo nell'ambiente senza la presenza di altri agenti

**Esempio 2.5.** Prendiamo ad esempio i seguenti ambienti provando a classificarli:

| | Crossword | Robo-selector | Poker | Taxi |
|---|---|---|---|---|
| Osservabile | Sì | Parziale | Parziale | Parziale |
| Deterministico | Sì | No | No | No |
| Episodico | No | Sì | No | No |
| Statico | Sì | No | Sì | No |
| Discreto | Sì | No | Sì | No |
| Singolo agente | Sì | Sì | No | No |

Il tipo di ambiente cambia radicalmente la soluzione del problema:

- **Deterministico, completamente osservabile**: Single-state problem
- **Completamente non osservabile**: Conformant problem, l'agente non sa in che stato si trova, ma potrebbe trovare una soluzione
- **Non deterministico e/o parzialmente osservabile**: Contingency problem, l'agente deve prevedere le possibili situazioni future e agire di conseguenza
- **Spazio degli stati sconosciuto**: Exploration problem, l'agente deve esplorare l'ambiente per scoprire gli stati e le azioni disponibili

## 2.4 Agenti di problem solving

È una forma ristretta di agenti che formulato un problema e un obiettivo partendo da uno stato cerca una soluzione ignorando le percezioni, siccome ci si trova in un single-state problem. Questo si chiama Offline problem solving perchè l'agente ha completa conoscenza dell'ambiente. Online problem solving è quando l'agente non ha completa conoscenza dell'ambiente.

**Esempio 2.6.** Il seguente è un esempio di problem solving agent:

```
function Simple-Problem-Solving-Agent(percept) returns action
  static: seq, an action sequence, initially empty
          state, some description of the current world state
          goal, a goal, initially null
          problem, a problem formulation

  state <- Update-State(state, percept)

  if seq is empty then
    goal <- Formulate-Goal(state)
    problem <- Formulate-Problem(state, goal)
    seq <- Search( problem)

  action <- First(seq)
  seq <- Rest(seq)
  return action

```

**Esempio 2.7.** Consideriamo il problema "Vacanze in Romania". Bisogna formulare un viaggio da Arad a Bucarest sapendo che l'aereo parte domani.

- **Goal**: Arrivare a Bucarest
- **Formulazione del problema**:
  - Stati: città della Romania
  - Azioni: volare tra le città
- **Soluzione**: Sequenza di città

Si potrebbe usare una mappa per trovare il percorso più breve (visione completa del mondo) e trovare una soluzione ottimale. Questo problema è definito da 4 componenti:

- **Stato iniziale**: ad esempio "ad Arad"
- **Funzione di transizione**: insieme di coppie (stato, azione) che mappano uno stato in un altro, ad esempio:

$$S(A) = \{\langle \text{Arad} \to \text{Zerind}, \text{Zerind} \rangle, \ldots\}$$

- **Test dell'obiettivo**: una funzione che verifica se lo stato corrente soddisfa l'obiettivo, ad esempio:

$$\text{Goal-Test}(s) = \begin{cases} \text{true} & \text{se } s = \text{Bucarest} \\ \text{false} & \text{altrimenti} \end{cases}$$

- **Path cost**: è una funzione che assegna un costo (additivo) a ogni azione, ad esempio la somma di distanze o il numero di azioni:

$$c(x, a, y) \geqslant 0$$

- **Soluzione**: Una sequenza di azioni che portano dallo stato iniziale allo stato obiettivo.

# 3 Ricerca nello spazio degli stati

## 3.1 Ricerca generale

### 3.1.1 Tree search

Un algoritmo di ricerca ad albero esplora lo spazio degli stati partendo dallo stato iniziale e generando nuovi stati (successori) applicando le azioni disponibili, cioè **espandendo** gli stati:

```
function Tree-Search(problem, strategy) runction Tree-Search(
    problem, strategy) returns a solution, or failure
initialize the search tree using the initial state of problem
loop do
if no candidates for expansion then return failure
choose a leaf node for expansion according to strategy
if node contains a goal state then return the solution
else add successor nodes to the search tree (expansion)
endeturns a solution, or failure
  initialize the search tree using the initial state of problem
  loop do
    if no candidates for expansion then return failure
      choose a leaf node for expansion according to strategy
    if node contains a goal state then return the solution
      else add successor nodes to the search tree (expansion)
end
```

### 3.1.2 Stato e nodo

Stato e nodo non sono la stessa cosa, infatti:

- **Stato**: rappresenta una configurazione dell'ambiente
- **Nodo**: è una struttura dati che costituisce una parte dell'albero di ricerca e include informazioni aggiuntive come il genitore, l'azione che ha portato a quello stato, il costo del percorso o la profondità nell'albero, ecc...

### 3.1.3 Tree search generale

Espandere un nodo significa generare i suoi figli, cioè i nodi successori e tutti i nodi non esplorati sono chiamati **frontiera**.

```
function Tree-Search( problem, frontier) returns a solution, or
    failure
  frontier <- Insert(Make-Node(problem.Initial-State))
  while not IsEmty(frontier) do
    node <- Pop(frontier)
    if problem.Goal-Test(node.State) then return node
```

```
      frontier <- InsertAll(Expand(node, problem))
  end loop
  return failure
```

La strategia è quella di scegliere l'ordine in cui i nodi vengono espansi, cioè come viene gestita la frontiera. Le strategie sono valutate in base a:

- **Completezza**: se garantisce di trovare una soluzione quando esiste
- **Complessità di tempo**: numero di nodi generati o espansi
- **Complessità di spazio**: numero massimo di nodi memorizzati in memoria
- **Ottimalità**: se garantisce di trovare la soluzione migliore

Le complessità di spazio e di tempo sono misurate in termini di:

- b: maximum branching factor, numero massimo di figli per nodo
- d: profondità della soluzione meno costosa
- m: profondità massima dell'albero di ricerca (potrebbe essere infinita)

### 3.1.4 Stati ripetuti

Fallire nel riconoscere stati ripetuti può trasformare un problema lineare in un problema esponenziale. Bisogna quindi mantenere una lista di stati già visitati e non espandere nodi che portano a stati già visitati:

```
function Graph-Search( problem, frontier) returns a solution, or
    failure
  explored <- an empty set
  frontier <- Insert(Make-Node(problem.Initial-State))
  while not IsEmty(frontier) do
    node <- Pop(frontier)
    if problem.Goal-Test(node.State) then return node
    if node.State is not in explored then
      add node.State to explored
      frontier <- InsertAll(Expand(node, problem))
    end if
  end loop
  return failure
```

## 3.2 Ricerca non informata

Gli algoritmi di ricerca non informata utilizzano soltanto i dati disponibili nella definizione del problema e i principali sono:

- Breadth-first search
- Uniform-cost search (Dijkstra)
- Depth-first search
- Depth-limited search
- Iterative deepening search

### 3.2.1 Breadth-first search

Questo algoritmo espande il nodo non esplorato più superficiale, cioè il nodo più vicino alla radice. Utilizza una coda FIFO per la frontiera e i nuovi successori vengono aggiunti alla fine della coda.

```
function BFS( problem) returns a solution, or failure
  node <- node with State=problem.Initial-State,Path-Cost=0
  if problem.Goal-Test(node.State) then return node
  explored <- empty set frontier <- FIFO queue with node as the
    only element
  loop do
    if frontier is empty then return failure
    node <- Pop(frontier)
    add node.State to explored
    for each action in problem.Actions(node.State) do
      child <- Child-Node(problem,node,action)
      if child.State is not in (explored or frontier) then
        if problem.Goal-Test(child.State) then return child
        frontier <- Insert(child)
      end if
    end for
  end loop
```

Questo tipo di ricerca è:

- **Completa**: Sì, soltanto se $b$ è finito, cioè se il branching factor è limitato
- **Complessità di tempo**: $b + b^2 + b^3 + \ldots + b^d = O(b^d)$
- **Complessità di spazio**: $O(b^d)$, perchè bisogna memorizzare tutti i nodi generati
- **Ottimale**: Sì, soltanto se il costo delle azioni è uniforme

### 3.2.2 Uniform-cost search

Questo algoritmo espande il nodo non esplorato con il **costo del percorso più basso**. La frontiera è una coda di priorità ordinata in base al costo del percorso. Questo tipo di ricerca è:

- **Completa**: Sì, se il costo minimo delle azioni $\geqslant \varepsilon$ (con piccola ma $\varepsilon > 0$)
- **Complessità di tempo**: Numero di nodi $g \leqslant$ del costo del percorso ottimale $C^*$. $O(b^{1+\lfloor C^*/\epsilon \rfloor})$
- **Complessità di spazio**: $O(b^{1+\lfloor C^*/\epsilon \rfloor})$
- **Ottimale**: Sì perchè i nodi vengono espansi in ordine di costo del percorso

Ci sono due modifiche principali rispetto alla BFS che garantiscono l'ottimalità:

1. Il goal test viene fatto quando il nodo viene estratto dalla frontiera, non quando viene generato. (Questo elemento spiega il $+1$ nella complessità
2. Controllare se un nodo generato è già presente nella frontiera con un costo più alto e in tal caso sostituirlo con il nuovo nodo a costo più basso

### 3.2.3 Depth-first search

Questo algoritmo espande il nodo non esplorato più profondo, cioè il nodo più lontano dalla radice. Utilizza una pila LIFO per la frontiera e i nuovi successori vengono aggiunti all'inizio. Questo tipo di ricerca è:

- **Completa**: No, perchè può rimanere bloccata in un ramo infinito, a meno che l'albero di ricerca non abbia una profondità limitata. Si potrebbero evitare loop modificando l'algoritmo per evitare stati ripetuti sul percorso corrente
- **Complessità di tempo**: $O(b^m)$, dove $m$ è la profondità massima dell'albero di ricerca
- **Complessità di spazio**: $O(bm)$, bisogna memorizzare soltanto il percorso corrente e i nodi fratelli
- **Ottimale**: No, perchè non garantisce di trovare la soluzione migliore

### 3.2.4 Iterative deepening search

Questo algoritmo combina i vantaggi della BFS e della DFS. Esegue una serie di ricerche in profondità limitata, aumentando progressivamente il limite di profondità fino a trovare una soluzione.

```
# Depth-Limited Search
function DLS(problem, limit) returns soln/fail/cutoff
  R-DLS(Make-Node(problem.Initial-State), problem, limit)


function R-DLS(node, problem, limit) returns soln/fail/cutoff
  if problem.Goal-Test(node.State) then return node
  else if limit = 0 then return cutoff # raggiunta la profondita'
    massima
  else
    # flag: c'e' stato un cutoff in uno dei sottoalberi?
    cutoff-occurred? <- false
    for each action in problem.Actions(node.State) do
      child <- Child-Node(problem, node, action)
      result <- R-DLS(child, problem, limit-1)
      if result = cutoff then cutoff-occurred? <- true
      else if result 6 = failure then return result
    end for
    if cutoff-occurred? then return cutoff else return failure
  end else

# Iterative Deepening Search
function IDS(problem) returns a solution
  inputs: problem, a problem
  for depth <- 0 to infinity do
    result <- DLS(problem, depth)
    if result 6 = cutoff then return result
  end
```

Questo tipo di ricerca è:

- **Completa**: Sì
- **Complessità di tempo**: $db^1 + (d-1)b^2 + \ldots + b^d = O(b^d)$
- **Complessità di spazio**: $O(bd)$

- **Ottimale**: Sì, se il costo delle azioni è uniforme

**Esercizio 3.1.** Assumi:

1. Un albero di ricerca ben bilanciato, tutti i nodi hanno lo stesso numero di figli
2. Il goal state è l'ultimo che viene espanso nel suo livello (il più a destra)
3. Se il branching factor è 3, la soluzione più superficiale è a profondità 3 (la radice è a profondità 0) e si utilizza la ricerca in ampiezza quanti nodi vengono generati?
4. Se il branching factor è 3, la soluzione più superficiale è a profondità 3 (la radice è a profondità 0) e si utilizza la iterative deepening quanti nodi vengono generati?

**Esercizio 3.2.** Un uomo ha un lupo, una pecora e un cavolo. L'uomo è sulla riva di un fiume con una barca che può trasportare solo lui e un altro oggetto. Il lupo mangia la pecora e la pecora mangia il cavolo, quindi non può lasciarli insieme da soli.

1. Formalizza il problema come un problema di ricerca
2. Usa BFS per risolvere il problema

**Soluzione:**
Formalizziamo gli stati come una tupla:

$$< W, S, C, M, B >$$

dove:

- $W$: posizione del lupo
- $S$: posizione della pecora
- $C$: posizione del cavolo
- $M$: posizione dell'uomo
- $B$: stato della barca

La posizione può essere 0 (left) o 1 (right).
Lo stato iniziale è:

$$< 0, 0, 0, 0, 0 >$$

Lo stato obiettivo è:

$$< 1, 1, 1, 1, 1 >$$

Le azioni possibili sono:

- Porta il lupo (CW)
- Porta la pecora (CS)
- Porta il cavolo (CC)
- Porta niente (CN)

| Operatore | Precondizione | Funzione |
|---|---|---|
| CW | $M = B, M = W, S \neq C$ | $\langle W, S, C, M, B\rangle \mapsto \langle \bar{W}, S, C, \bar{M}, \bar{B}\rangle$ |
| CS | $M = B, M = S$ | $\langle W, S, C, M, B\rangle \mapsto \langle W, \bar{S}, C, \bar{M}, \bar{B}\rangle$ |
| CC | $M = B, M = C, W \neq S$ | $\langle W, S, C, M, B\rangle \mapsto \langle W, S, \bar{C}, \bar{M}, \bar{B}\rangle$ |
| CN | $M = B$ | $\langle W, S, C, M, B\rangle \mapsto \langle W, S, C, \bar{M}, \bar{B}\rangle$ |

Notiamo che in tutte le precondizioni c'è $M = B$ perchè l'uomo deve essere sempre con la barca, quindi si possono unire i due stati in uno solo $M$.

## 3.3 Ricerca informata

Gli algoritmi di ricerca informata utilizzano informazioni aggiuntive (euristiche) per guidare la ricerca verso la soluzione in modo più efficiente.

### 3.3.1 Best-first search

Questo algoritmo usa una **funzione di valutazione** per ogni nodo che stima la "desiderabilità". La frontiera è una coda ordinata in ordine decrescente di desiderabilità. A seconda di come viene definita la desiderabilità si ottengono diversi algoritmi:

- Greedy best-first search
- A*

### 3.3.2 Greedy best-first search

Questo algoritmo espande il nodo che sembra essere il più vicino alla soluzione secondo una funzione di valutazione euristica $h(n)$ che stima il costo rimanente per raggiungere l'obiettivo da un nodo $n$.

**Esempio 3.1.** In una mappa di una città, la funzione di valutazione potrebbe essere la distanza in linea d'aria dal nodo corrente alla destinazione. In questo modo, l'algoritmo esplora prima i nodi che sembrano più vicini alla destinazione, riducendo il numero di nodi esplorati rispetto a una ricerca non informata.

Questo tipo di ricerca è:

- **Completa**: No, perchè può rimanere bloccata in un ciclo infinito. È completo se lo spazio di ricerca è finito e ci sono controlli per evitare stati ripetuti
- **Complessità di tempo**: $O(b^m)$ nel peggiore dei casi, ma può essere molto più veloce con una buona euristica

- **Complessità di spazio**: $O(b^m)$, bisogna memorizzare tutti i nodi generati
- **Ottimale**: No

### 3.3.3 A* search

Questo algoritmo evita di espandere cammini che sono già molto costosi e ha come funzione di valutazione:

$$f(n) = g(n) + h(n)$$

dove:

- $g(n)$: costo del percorso dal nodo iniziale a $n$
- $h(n)$: stima del costo rimanente per raggiungere l'obiettivo da $n$
- $f(n)$: stima del costo totale del percorso passando per $n$

L'euristica, per poter garantire l'ottimalità, deve essere **ammissibile**, cioè per ogni nodo la stima di quel nodo deve essere minore o uguale del vero costo per arrivare all'obbiettivo, quindi non deve **sovrastimare** il costo rimanente:

$$h(n) \leqslant h^*(n) \quad h(n) \geqslant 0 \rightarrow h(G) = 0$$

dove $h^*(n)$ è il costo effettivo del percorso da $n$.

**Teorema 3.1.** Per A* l'euristica ammissibile implica l'ottimalità

Questo tipo di ricerca è:

- **Completa**: Sì, tranne se ci sono infiniti nodi con $f \leqslant f(G)$
- **Complessità di tempo**: Esponenziale in errore relativo in $h\times$ lunghezza del numeo di passi della soluzione ottimale. (Se l'euristica è buona, la complessità sarà molto più bassa)
- **Complessità di spazio**: $O(b^d)$, bisogna memorizzare tutti i nodi generati
- **Ottimale**: Sì, ma richiede assunzioni sull'euristica (ammissibilità, consistenza) e una strategia di ricerca (ricerca ad albero o grafo)

### 3.3.4 Consistenza e ammissibilità

**Definizione 3.1.** Un euristica è **consistente** se:

$$h(n) \leqslant c(n, a, n') + h(n')$$

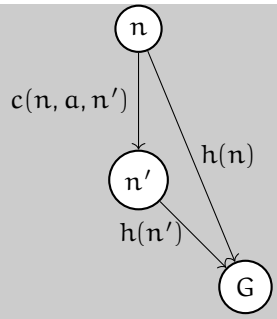



Figura 1: Esempio di euristica consistente

- Si può dimostrare che se $h$ è consistente, allora $f(n)$ non decresce lungo qualsiasi cammino
- A* espande i nodi in ordine crescente di $f$, quindi trova sempre la soluzione ottimale

Quindi si espande sempre prima un cammino ottimo rispetto a un cammino non ottimo.

La consistenza implica l'ammissibilità e può essere dimostrato per induzione sul cammino verso il goal. L'ammissibilità però non implica la consistenza.

$$\text{Consistenza} \rightarrow \text{Ammissibilità}$$
$$\text{Ammissibilità} \not\rightarrow \text{Consistenza}$$

- Tree-Search + euristica ammissibile $\rightarrow$ A* ottimale
- Graph-Search + euristica ammissibile $\not\rightarrow$ A* ottimale (può scartare il cammino ottimale per un nodo ripetuto)
- Graph-Search + euristica consistente $\rightarrow$ A* ottimale

### 3.3.5 Euristiche

Le euristiche possono essere create in diversi modi, prendiamo ad esempio l'8-puzzle:

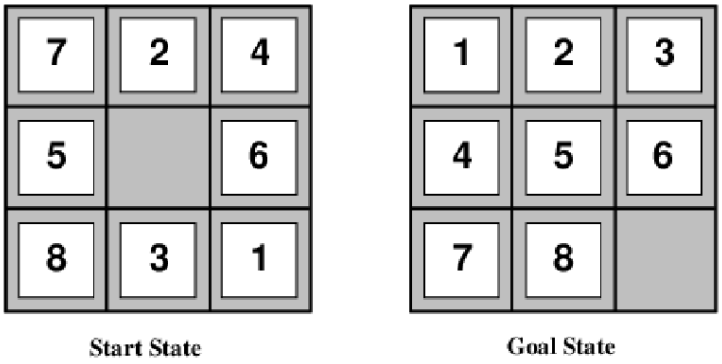



Figura 2: Esempio di 8-puzzle

Per questo problema si potrebbe utilizzare come euristica:

- $h_1(n)$ = numero di pezzi fuori posto
- $h_2(n)$ = somma delle distanze di Manhattan (numero di mosse orizzontali e verticali necessarie per portare ogni pezzo alla sua posizione obiettivo)

Entrambe le euristiche sono ammissibili, ma $h_2$ è più precisa di $h_1$ perchè fornisce una stima più vicina al costo reale per raggiungere l'obiettivo.

In questo caso si dice che $h_2$ **domina** $h_1$ se sono entrambe ammissibili e $h_2(n)$ è sempre maggiore o uguale a $h_1$:

$$h_2(n) \geqslant h_1(n) \quad \forall n$$

**Teorema 3.2.** Date due qualsiasi euristiche **ammissibili** $h_a$ e $h_b$, allora l'euristica definita come:

$$h(n) = \max(h_a(n), h_b(n))$$

è anch'essa ammissibile e domina sia $h_a$ che $h_b$

Le euristiche ammissiibli possono essere derivate dall'esatto costo della soluzione di un problema **rilassato**, cioè un problema simile a quello originale ma con restrizioni rimosse. Ad esempio, per l'8-puzzle si potrebbe rilassare il problema permettendo di muovere una casella ovunque (in questo caso $h_1(n)$ da la soluzione migliore), oppure permettendo di muovere una casella in qualsiasi casella adiacente ( in questo caso $h_2(n)$ da la soluzione migliore)

**Definizione utile 3.1.** Il costo della soluzione ottimale di un problema rilassato non è maggiore del costo della soluzione ottimale del problema reale.

## 3.4 Ricerca locale

La ricerca locale è una tecnica di ricerca che si concentra su una soluzione cercando di migliorarla iterativamente. Gli algoritmi più comuni sono:

- Hill climbing
- Simulated annealing
- Algoritmi genetici

Questo tipo di ricerca è utile quando il percorso per arrivare alla soluzione non è importante, ma solo la soluzione finale. Ci sono due approcci principali:

- Trovare la configurazione ottimale (ad esempio (TSP - Traveling Salesman Problem))
- Trovare una configurazione che soddisfa dei vincoli (ad esempio il problema delle $n$ regine)

Si possono anche usare algoritmi di **iterative improvement** che partono da una configurazione iniziale e cercano di migliorarla iterativamente fino a raggiungere un punto di ottimo locale.

**Esempio 3.2.** Un esempio di ricerca locale è il Traveling Salesman Problem (TSP), dove l'obiettivo è trovare il percorso più breve che visita un insieme di città esattamente una volta e ritorna alla città di partenza.

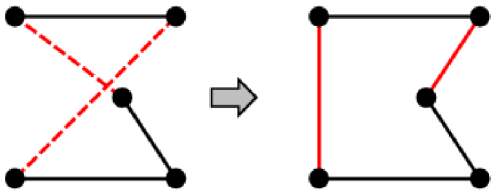


Figura 3: Esempio di TSP

Varianti di questo approccio arrivano fino a 1% della soluzione ottimale in tempi ragionevoli per migliaia di città.

**Esempio 3.3.** Un altro esempio è il problema delle $n$ regine, dove l'obiettivo è posizionare $n$ regine su una scacchiera $n \times n$ in modo che nessuna regina minacci un'altra.

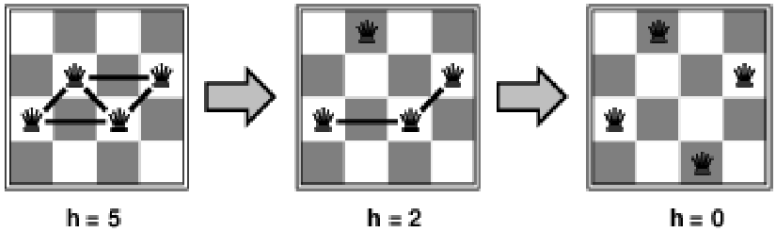



Figura 4: Esempio di n-regine

Risolve quasi sempre il problema quasi istantaneamente per $n$ molto grande, ad esempio $n = 1.000.000$.

### 3.4.1 Hill climbing

Hill climbing è un algoritmo di ricerca locale che parte da una soluzione iniziale e cerca di migliorarla iterativamente spostandosi verso la soluzione migliore nella sua vicinanza. L'algoritmo continua a muoversi finché non trova un punto dove nessuna delle soluzioni vicine è migliore della soluzione corrente. Ritorna un massimo locale, che potrebbe non essere il massimo globale.

```
function Hill-Climbing(problem) returns a state that is a local
    maximum
  inputs: problem, a problem
  local variables: current, a node
                   neighbor, a node
  current <- Make-Node(problem.Initial-State)
```

```
  loop do
    neighbor <- a highest-valued successor of current
    if neighbour.Value <= current.Value then return
      current.State
    end if
    current <- neighbor
  end
```

La rappresentazione del problema assumendo di avere tutti gli stati sull'asse x e il valore della funzione obiettivo sull'asse y è chiamata **state space landscape**:

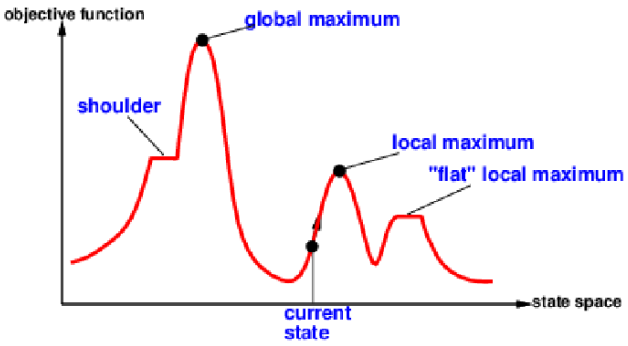



Figura 5: Esempio di state space landscape per hill climbing

- **Random-restart hill climbing**: esegue hill climbing più volte da stati iniziali casuali per aumentare le probabilità di trovare il massimo globale
- **Random sideways moves**: permette di fare mosse che non migliorano la soluzione corrente per evitare di rimanere bloccati in una sezione piatta

### 3.4.2 Simulated annealing

Simulated annealing è un algoritmo di ricerca locale ispirato al processo di raffreddamento dei metalli. L'algoritmo parte da una soluzione iniziale e cerca di migliorarla iterativamente, ma a differenza di hill climbing, permette di **accettare soluzioni peggiori** con una certa probabilità che **diminuisce nel tempo**. Questo aiuta a evitare di rimanere bloccati in massimi locali.

```
function Simulated-Annealing( problem, schedule) returns a solution
     state
  inputs: problem, a problem
          schedule, a mapping from time to 'temperature'
  local variables: current, a node
                   next, a node
                   T, a 'temperature' controlling prob. of downward
     steps
  current <- Make-Node(problem.Initial-State)
  for t <- 1 to infinity do
    T <- schedule(t) // temperature at time t
    if T = 0 then return current
    next <- a randomly selected successor of current
    deltaE <- next.Value - current.Value
    if deltaE > 0 then current <- next
    else current <- next only with probability e^(deltaE/T)
```

Se la "temperatura" T diminuisce lentamente abbastanza, allora l'algoritmo converge **sempre** alla soluzione ottimale $x^*$ . Questo perchè:

$$e^{\frac{E(x^*)}{kT}} / e^{\frac{E(x)}{k}T} = e^{\frac{E(x^*)-E(x)}{kT}} \gg 1 \quad \text{per } T \to 0$$

### 3.4.3 Local beam search

Local beam search è un algoritmo di ricerca locale che mantiene **un insieme di soluzioni candidate** e cerca di migliorarle iterativamente. In ogni iterazione, l'algoritmo **seleziona casualmente i successori di** $k$**, ma con un bias verso i migliori** per formare il nuovo insieme di soluzioni candidate.

### 3.4.4 Algoritmi genetici

Gli algoritmi genetici sono una classe di algoritmi di ricerca ispirati ai processi di evoluzione biologica. Questi algoritmi utilizzano meccanismi simili alla selezione naturale, alla mutazione e al crossover per evolvere una popolazione di soluzioni candidate verso soluzioni migliori. Gli algoritmi genetici hanno bisogno di stati encodati come stringhe di caratteri. Il crossover combina due stringhe per creare una nuova stringa e ha senso solo se le sottostringhe hanno un significato indipendente.

**Esempio 3.4.** Prendiamo ad esempio il problema delle $n$ regine in cui si ha un encoding della scacchiera come un numero in cui la cifra in posizione $i$ rappresenta la riga in cui si trova la regina nella colonna $i$.

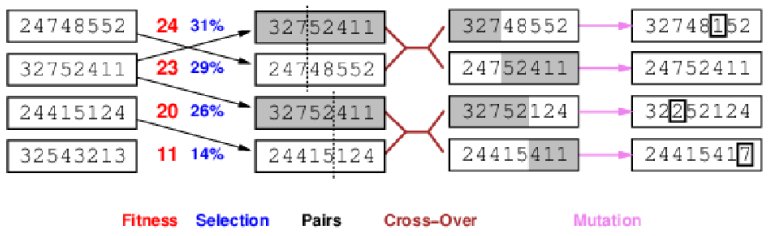



Figura 6: Esempio di algoritmo genetico per il problema delle n-regine

Questo genera una configurazione nuova a partire da coppie di configurazioni esistenti, selezionate in base alla loro "fitness" (un valore che misura quanto una configurazione si avvicina alla soluzione ottimale).

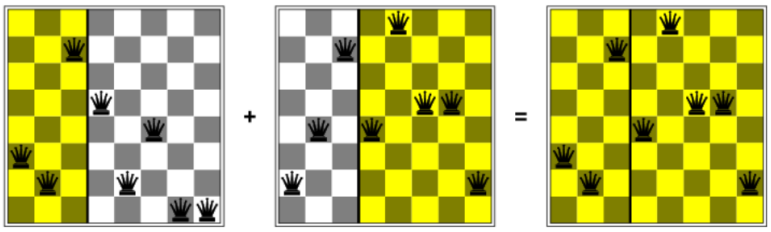


Figura 7: Rappresentazione della scacchiera

In questo caso la fitness è calcolata come il numero di regine che si minacciano a vicenda. Il caso peggiore è quello in cui tutte le regine si minacciano a vicenda, quindi la fitness è 28 (per $n = 8$) perchè:

$$\text{num_minacce} = \frac{n(n-1)}{2} = \frac{8 \cdot 7}{2} = 28$$

quindi:

$$\text{fitness} = 28 - \text{num_minacce}$$

## 3.5 Ricerca locale in uno spazio continuo

La ricerca locale può essere estesa a spazi di stato continui. Per risolvere questi problemi si possono utilizzare tecniche come:

- **Discretizzazione**: suddividere lo spazio continuo in una griglia di punti discreti con una risoluzione $\delta$ e utilizzare algoritmi di ricerca locale
- **Random perturbations**: si prende una soluzione e si applicano piccole perturbazioni casuali per esplorare lo spazio delle soluzioni
- **Gradient**: si calcola analiticamente il gradiente della funzione obiettivo $f(x)$

### 3.5.1 Gradient ascent/descent

Il gradiente di una funzione $f(x)$ è il seguente:

$$\nabla f(x) = \left( \frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \ldots, \frac{\partial f}{\partial x_n} \right)$$

Per trovare la direzione di massima crescita della funzione obiettivo si pone il gradiente uguale a zero:

$$\nabla f(x) = 0$$

Spesso non si può porre il gradiente a 0 globalmente, ma si può migliorare localmente:

- Aggiornare la soluzione nella direzione massima del gradiente per ogni coordinata
- Più la funzione è "ripida" più si fanno passi grandi

Aggiornare una coordinata viene effettuato tramite una funzione generale $g(x_1, x_2)$:

$$x_1 \leftarrow x_1 + \alpha \frac{\partial g(x_1, x_2)}{\partial x_1} \quad x_2 \leftarrow x_2 + \alpha \frac{\partial g(x_1, x_2)}{\partial x_2}$$

Oppure in forma vettoriale:

$$X = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}, \quad \nabla g(X) = \begin{bmatrix} \frac{\partial g(X)}{\partial x_1} \\ \frac{\partial g(X)}{\partial x_2} \end{bmatrix}$$

$$X \leftarrow X + \alpha \nabla g(X)$$

Dove $\alpha$ è lo "step size", cioè la dimensione del passo da fare:

- Se $\alpha$ è troppo grande si rischia di saltare soluzioni
- Se $\alpha$ è troppo piccolo i passi richiesti possono essere troppi

### 3.5.2 Algoritmo di Newton-Raphson

È una tecnica generale per trovare le radici di una funzione, cioè risolvere un'equazione del tipo $g(x) = 0$. Per farlo si trova un'approssimazione iniziale $\bar{x}_0$ della soluzione e iterativamente si aggiorna l'approssimazione usando la formula:

$$\bar{x}_{n+1} = \bar{x}_n - \frac{g(\bar{x}_n)}{g'(\bar{x}_n)}$$

dove $g'$ è la derivata di $g$:

$$g'(x) = \frac{dg(x)}{dx}$$

**Esempio 3.5.** Consideriamo la funzione $g(x) = x^2 - a$.

1. Mostra che il metodo di Newton-Raphson porta alla formula:

$$x_{n+1} = \frac{1}{2}\left(x_n + \frac{a}{x_n}\right)$$

**Soluzione:**
La formula di Newton-Raphson è:

$$\begin{aligned} \bar{x}_{n+1} &= \bar{x}_n - \frac{g(\bar{x}_n)}{g'(\bar{x}_n)} \\ &= \bar{x}_n - \frac{\bar{x}_n^2 - a}{2\bar{x}_n} \\ &= \frac{2\bar{x}_n^2 - (\bar{x}_n^2 - a)}{2\bar{x}_n} \\ &= \frac{\bar{x}_n^2 + a}{2\bar{x}_n} \\ &= \frac{1}{2}\left(\bar{x}_n + \frac{a}{\bar{x}_n}\right) \end{aligned}$$

2. Fissato $a = 4$ e $x_0 = 1$, calcola $x_i$, $i \in \{1, 2, 3\}$

**Soluzione:**
Sostituendo i valori nella formula ottenuta al passo precedente si ottiene:

$$x_1 = \frac{1}{2}\left(1 + \frac{4}{1}\right) = \frac{5}{2} = 2.5$$
$$x_2 = \frac{1}{2}\left(\frac{5}{2} + \frac{4}{\frac{5}{2}}\right) = \frac{1}{2}\left(\frac{25}{10} + \frac{8}{5}\right) = \frac{1}{2} \cdot \frac{41}{10} = \frac{41}{20} = 2.05$$
$$x_3 = \frac{1}{2}\left(\frac{41}{20} + \frac{4}{\frac{41}{20}}\right) = \frac{1}{2}\left(\frac{41}{20} + \frac{80}{41}\right) = \frac{1}{2} \cdot \frac{1681 + 1600}{820} \approx 2.0006$$

Graficamente si può vedere che la funzione converge rapidamente a 2:

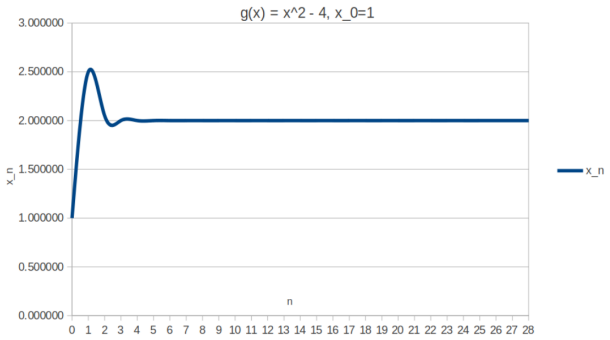



Figura 8: Esempio grafico del metodo di Newton-Raphson

### 3.5.3 Calcolo degli zeri del gradiente

Utilizzando il metodo di Newton-Raphson si possono trovare gli zeri del gradiente, cioè dove la funzione generica $g(x)$ è $\nabla f(X)$. In questo caso le funzioni di aggiornamento in forma vettoriale diventano:

$$x \leftarrow x - H_f^{-1}(x)\nabla f(x)$$

dove $H_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}$ è la matrice Hessiana. Per problemi in più dimensioni calcolare tutte le entrate della matrice Hessiana può essere computazionalmente costoso, quindi spesso si usano metodi approssimati.

Questo è ancora un metodo **locale**, quindi soffre degli stessi problemi della ricerca locale, come massimi locali e punti sella. Random restart e simulated annealing possono essere utili anche per spazi continui.

### 3.5.4 Gradiente empirico

A volte si può calcolare $f(X)$ per un certo input, ma non si può calcolare $\nabla f(X) = 0$ neanche localmente. L'**empirical gradient** è la risposta di $f(X)$ a piccoli incrementi o decrementi di $X$.

## 3.6 Constrained satisfaction problem

Assumiamo di avere un singolo agente, azioni deterministiche e un ambiente completamente osservabile (discreto). In questo tipo di problemi:

- Lo stato è definito da un insieme di variabili $X = X_i$ che può assumere valori in un insieme di domini $D = D_i$.
- Il goal test è un insieme di vincoli che specificano le combinazioni ammissibili per sottoinsiemi di variabili
- Si possono usare algoritmi che sfruttano queste proprietà che sono più efficenti degli algoritmi di ricerca generici

**Esempio 3.6.** Un esempio di problema CSP è il problema del map coloring, in cui si deve colorare una mappa in modo che nessuna regione confinante abbia lo stesso colore.

- **Variabili**: $WA, NT, Q, NSW, V, SA, T$
- **Domini**: $D_i = \{red, green, blue\}$
- **Vincoli**: Regioni adiacenti devono avere colori diversi. Si rappresenta con: $WA \neq NT$




Figura 9: Esempio di map coloring

Una soluzione è ad esempio:

$$\{WA = red, NT = green, Q = red, NSW = green, V = red, SA = blue, T = green\}$$

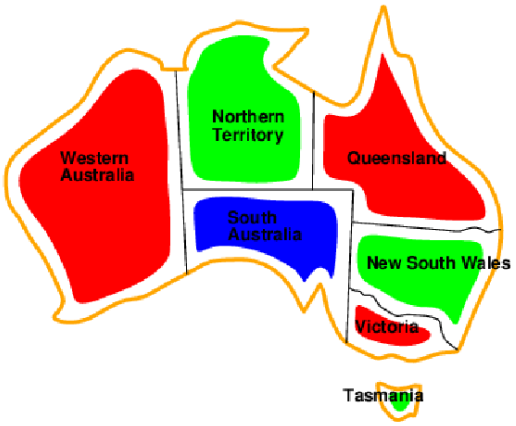



Figura 10: Esempio di soluzione del map coloring

### 3.6.1 Grafo dei vincoli

Il grafo dei vincoli, detto anche primal graph, è una rappresentazione grafica di un problema CSP in cui è presente un nodo per ogni variabile e un arco per ogni vincolo tra due variabili:

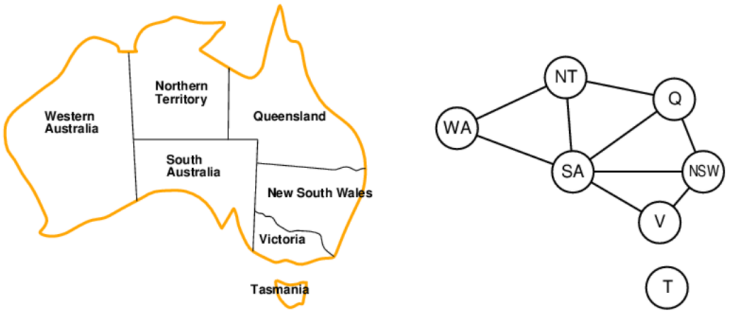



Figura 11: Esempio di grafo dei vincoli per il problema del map coloring

**Definizione 3.2.** Il grafo dei vincoli è definito come una tupla di 3 elementi:

$$CN = \langle X, D, C \rangle$$

dove:

- $X = \{x_1, \ldots, x_n\}$: insieme di variabili
- $D = \{D_1, \ldots, D_n\}$: insieme di domini
- $C = \{(S_1, R_1), \ldots, (S_m, R_m)\}$: insieme di vincoli, dove ogni vincolo $(S_i, R_i)$ è composto da:

- $S_i \subseteq X$: sottoinsieme di variabili coinvolte nel vincolo (scope)
- $R_i$: sottoinsieme del prodotto cartesiano delle variabili in $S_i$, cioè l'insieme delle combinazioni ammissibili delle variabili in $S_i$

- Soluzione: un'assegnazione di **tutte** le variabili che soddisfa **tutti** i vincoli. Esistono anche soluzioni parziali consistenti, cioè una soluzione parziale che soddisfa tutti i vincoli in cui lo scope contiene solo variabili assegnate. La soluzione parziale consistente non è necessariamente parte di una soluzione completa.
- Tasks: è una funzione di ottimizzazione, ad esempio controllo di consistenza, trovare una o tutte le soluzioni

**Esempio 3.7.** Consideriamo seguente crossword:

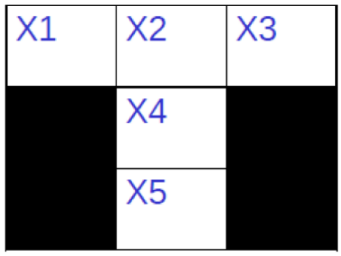



Figura 12: Esempio di crossword

Bisogna assegnare le lettere delle parole disponibili alle caselle vuote in modo che le parole risultanti siano valide.

- **Variabili**: parole possibili: MAP, ARC
- **Domini**: $D_i =$ lettere dell'alfabeto
- **Vincoli**: lettere condivise devono essere uguali:

$$C_1\,[\{x_1, x_2, x_3\}, (\text{MAP}), (\text{ARC})]$$
$$C_2\,[\{x_2, x_4, x_5\}, (\text{MAP}), (\text{ARC})]$$

Il grafo dei vincoli è il seguente:

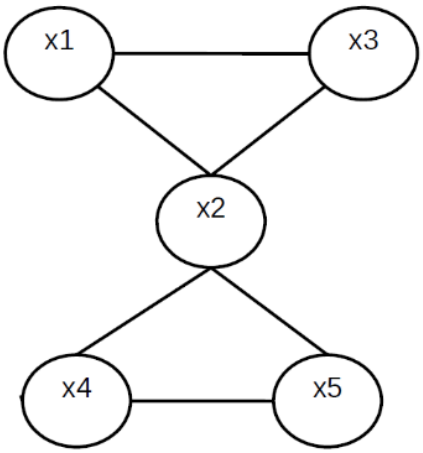



Figura 13: Esempio di grafo dei vincoli per il problema del crossword

Esiste un altro grafo chiamato **grafo duale** in cui i nodi rappresentano i vincoli e gli archi rappresentano le variabili condivise tra i vincoli (vincolo di uguaglianza):

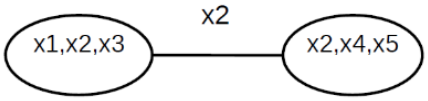



Figura 14: Esempio di grafo duale per il problema del crossword

### 3.6.2 Problemi combinatori

I problemi combinatori sono tutti quei problemi in cui dato un insisme di soluzioni si vuole trovare la soluzione migliore. Il CSP è un sottoinsieme di problemi combinatori. I principali tipi di problemi combinatori sono:

- **Decisioni**: dato un insieme di soluzioni, decidere se esiste una soluzione che soddisfa certi criteri. Ad esempio colora un grafo con k colori, bisogna dire se è possibile o no fissato un k.
- **Ottimizzazione**: bisogna ottimizzare un obiettivo. Ad esempio colorare un grafo con k colori minimizzando i conflutti.
- **Ottimizzazione multiobiettivo**: bisogna ottimizzare più obiettivi contemporaneamente. Ad esempio minimizzare il rischio e massimizzare il profitto in un portafoglio di investimenti.
- **Graphical models**: sono problemi definiti da:
  - Insieme di variabili
  - Domini delle variabili
  - Funzioni **locali** che definiscono i vincoli

- Funzione **globale** che rappresenta un'aggregazione delle funzioni locali
- Soluzioni, ovvero assegnazioni delle variabili, che ottimizzano la funzione globale

### 3.6.3 Backtracking search

Il backtracking search è un algoritmo di ricerca per i problemi CSP. Questo algoritmo è utile quando le assegnazioni delle variabili sono commutative, cioè l'ordine in cui le variabili vengono assegnate non cambia il risultato finale:

$$WA = \text{red}, \text{NT} = \text{green} \quad \text{è equivalente a} \quad \text{NT} = \text{green}, WA = \text{red}$$

Questo algoritmo è semplicemente una ricerca in profondità per CSP con assegnamenti alle variabili singoli. L'ordine delle variabili può impattare la performance dell'algoritmo.

```
function Backtracking-Search(csp) returns solution or failure
  return Backtrack({ }, csp)

function Backtrack(assignment, csp) returns solution or failure
  if assignment is complete then return assignment
  var <- Select-Unassigned-Variable(csp)
  for each value in Order-Domain-Values(var, assignment, csp) do
    if value is consistent with assignment then
      add {var = value} to assignment
      inferences <- Inferences(csp, var, value)
      if inferences 6 = failure then
        add inferences to assignment
        result <- Backtrack(assignment, csp)
        if result 6 = failure then
          return result
        endif
      endif
    endif
    remove {var = value} and inferences from assignment
  endfor
  return failure
```

Le principali decisioni che impattano l'algoritmo sono:

- Come selezionare la variabile
- Come selezionare il valore
- Come fare inferenze

Per migliorare l'algoritmo si possono usare le seguenti tecniche:

- **Ordinare le variabili**:
  - Minimum Remaining Values: Seleziona la variabile con il minor numero di valori legali rimasti nel dominio, quindi prima si fallisce meglio è



Figura 15: Esempio di Minimum Remaining Values

– Degree Heuristic: Seleziona la variabile che è coinvolta nel maggior numero di vincoli con altre variabili non assegnate



Figura 16: Esempio di Degree Heuristic

- **Ordinare i valori delle variabili**:
  - Least Constraining Value: Seleziona il valore che lascia il maggior numero di opzioni aperte per le altre variabili non assegnate




Figura 17: Esempio di Least Constraining Value

- **Consistenza locale**: Arc o Path consistency per ridurre i domini delle variabili e individuare fallimenti precoci
- **Look-ahead**: Predirre i conflitti futuri per evitare di esplorare rami che porteranno a fallimenti
- **Look-back**: Analizzare verso dove fare backtracking
- **Tree decomposition**: Sfrutta la struttura del problema per dividerlo in sotto-problemi più piccoli e risolverli separatamente

### 3.6.4 Inferenza

Si possono fare ragionamenti riguardo ai vincoli per fare inferenze per nuovi vincoli. Se consideriamo l'esempio del graph coloring:

- Dati $\{x_1, x_2, x_3\}$, $\{D_1 = D_2 = D_3\}, D_i = \{R, B\}$ e $C = \{C_1 : (x_1 \neq x_2), C_2 : (x_2 \neq x_3)\}$
- Si può inferire che $C_3 : (x_1 \neq x_3)$ perchè se $x_1 = x_3$ allora $x_2$ non può assumere nessun valore legale

I vantaggi e svantaggi dell'aggiunta di vincoli sono:

- Vincoli più stringenti portano ad uno spazio di ricerca più piccolo
- Aggiungere vincoli richiede più computazione
- Ogni volta che una nuova variabile viene assegnata bisogna controllare più vincoli

- Se il problema consiste soltanto in vincoli binari allora non si hanno mai più di $O(n)$ controlli
- Se il problema consiste in vincoli di ordine superiore $r$ allora si hanno $O(n^{r-1})$ controlli

Questa inferenza rende il grafo backtrack free.

Un grafo si dice **backtrack free** se ogni foglia è un goal state. Una DFS su un grafo backtrack-free garantisce un assegnamento completo e consistente

### 3.6.5 Look Ahead

Look ahead è una tecnica che permette di fare inferenze sui vincoli futuri per evitare di esplorare rami che porteranno a fallimenti. Quindi data un'inferenza approssimata si vuole predirre l'impatto della prossima assegnazione di una variabile e vedere come impatta i futuri assegnamenti. Ci sono due strategie principali:

- **Forward checking**: Controlla le variabili assegnate separatamente da quelle non assegnate.
- **Arc consistency look ahead**: Propaga la consistenza di arco, cioè quella che assicura che per ogni valore di una variabile esista un valore legale nella variabile connessa tramite un vincolo binario, in tutta la rete

### 3.6.6 Forward checking look ahead

È la forma più limitata di propagazione dei vincoli. Propaga l'effetto di un valore selezionato su tutte le variabili future **separatamente**, cioè una ad una. Se il dominio di una variabile futura diventa vuoto, allora si tenta il valore successivo per la variabile corrente.

**Esempio 3.8.** Prendiamo ad esempio il problema del map coloring, l'idea principale è quella di tenere traccia dei valori legali rimanenti per le variabili non assegnate. Viene terminata la ricerca quando qualsiasi variabile non ha più valori legali.

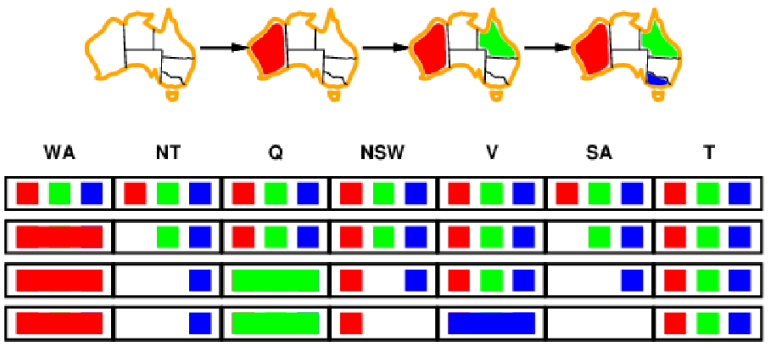



Figura 18: Esempio di forward checking per il problema del map coloring

La complessità del forward checking è: $O(ek^2)$, dove:

- $e$ il numero di vincoli
- $k$ valore per ogni variabile futura
- $k$ valore per la variabile corrente

### 3.6.7 Arc consistency look ahead

L'arc consistency look ahead forza la consistenza di arco su tutte le variabili rimanenti.

- Un arco $x_k \rightarrow x_j$ è **consistente** se e solo se per ogni assegnamento di $x_k$ c'è almeno un assegnamento di $x_j$ che è consistente con il vincolo $(x_k, x_j)$.
- Forzare la consistenza di arco: se nessun valore di $x_j$ è consistente con un dato valore di $x_k = c$ allora si rimuove $c$ dal dominio di $x_k$
- Forward checking: si impone la consistenza da ogni variabile non assegnata a un nuovo assegnamento
- Nota: nel forward checking non si controlla mai la consistenza tra variabili non assegnate, nell'arc consistency look-ahead per ogni nuovo assegnamento si controlla la consistenza di arco tra ogni coppia di variabili

L'arc consistency viene rappresentata da una funzione che modifica il dominio di $x_k$ se non è consistente con $x_j$:

$$rev(x_k, x_j)$$

**Esempio 3.9.** Consideriamo il problema del map coloring. I passi dell'algoritmo sono:

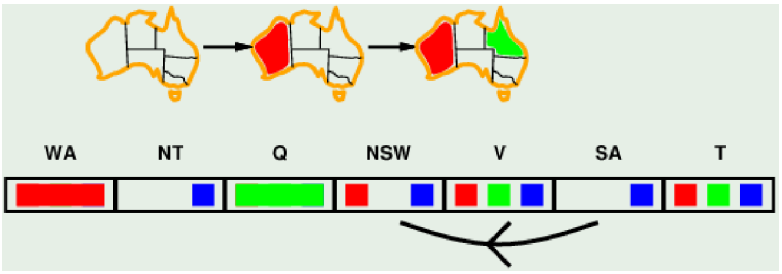



Figura 19: Passo 1

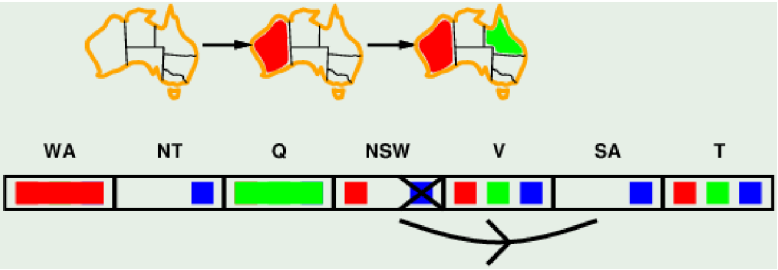



Figura 20: Passo 2

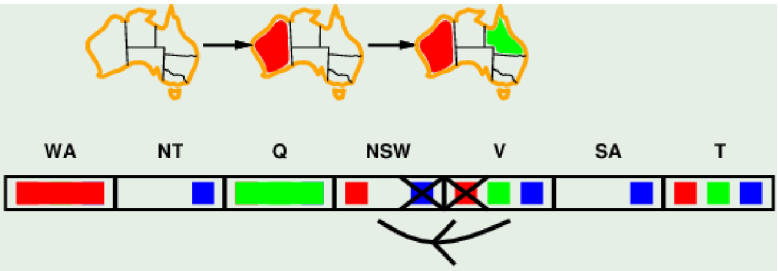



Figura 21: Passo 3

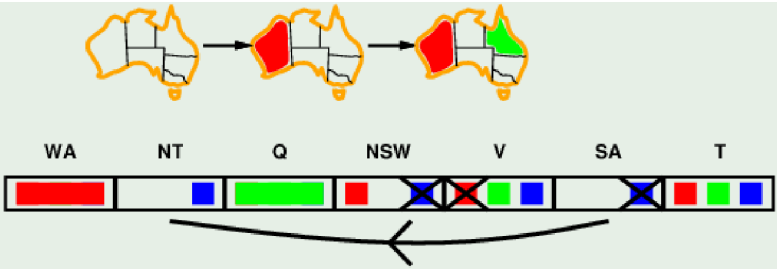



Figura 22: Passo 4

La complessità dell'algoritmo arc consistency migliore (AC-4) è $O(ek^3)$.

### 3.6.8 Forzare l'arc consistency

**Definizione 3.3.**

- Un arco $x_i, x_j$ è arc consistent se e solo se $x_i$ è arc consistent rispetto a $x_j$ e $x_j$ è arc consistent rispetto a $x_i$.
- Un grafo è arc consistent se e solo se tutti i suoi vincoli sono arc consistent.
- Si può facilmente assicurare che una coppia di variabili siano arc consistent (procedura Revise), Ma rivedere la consistenza di arco su una

variabile potrebbe rendere un'altra variabile non arc consistent.

- Sono necessarie procedure sistematiche per garantire l'arc consistency sulle reti.

- L'arc consistency con un dominio vuoto implica che il problema non ha soluzione.
- L'arc consistency con tutti i domini non vuoti **non** implica che ci sia una soluzione. Quindi l'arc consistency non è completa.
- L'unica cosa che si può dire a riguardo è che se l'arc consistency con tutti i domini non vuoti non ha cicli nel grafo dei vincoli e ha solo vincoli binari, allora il grafo è backtrack free ed esiste una soluzione.

### 3.6.9 Tree decomposition

Consiste nel decomporre il problema in una struttura ad albero che permette di sfruttare la proprietà degli alberi di essere backtrack free. L'idea più semplice è quella di togliere le variabili assegnate fino a che il grafo non diventa un albero.

**Definizione 3.4.** Dato un grafo non orientato, il sottoinsieme di nodi del grafo è definito **cycle cutset** se la rimozione di questi nodi rende il grafo aciclico.

Per trovare una soluzione al problema si devono provare tutte le possibili assegnazioni delle variabili nel cycle cutset e per ogni assegnazione si risolve il problema tramite arc propagation. La complessità è esponenziale, ma dipende solo dal numero di nodi nel cycle cutset.

# 4 Logical Agents

Un agente logico è un agente che utilizza la logica per rappresentare la conoscenza e ragionare su di essa. La logica fornisce un linguaggio formale per esprimere fatti e regole sul mondo, permettendo all'agente di dedurre nuove informazioni e prendere decisioni basate sulla conoscenza acquisita.

## 4.1 Knowledge based agents

Gli agenti knowledge based sono divisi in due componenti principali:

- **Inference engine**: si occupa di dedurre nuove informazioni dalla knowledge base utilizzando regole logiche
- **Knowledge base**: contiene la conoscenza dell'agente rappresentata in forma logica, cioè è un insieme di frasi logiche appartenenti ad un linguaggio formale

L'approccio dichiarativo per costruiere un agente consiste nel **dirgli** (Tell) cosa deve sapere e poi l'agente può **chiedere** (Ask) alla sua knowledge base per sapere cosa fare.

Un esempio di agente knowledge based è il seguente:

```
function KB-Agent( percept) returns an action
  static: KB, a knowledge base
          t, a counter, initially 0, indicating time
  Tell(KB, Make-Percept-Sentence( percept, t))
  action <- Ask(KB, Make-Action-Query(t))
  Tell(KB, Make-Action-Sentence(action, t))
  t <- t + 1
  return action
```

L'agente deve essere in grado di:

- Rappresentare stati, azioni ecc...
- Incorporare nuove percezioni nella knowledge base
- Aggiornare le rappresentazioni interne del mondo
- **Dedurre** proprietà nascoste del mondo
- **Dedurre** azioni appropriate

**Esempio 4.1.** Consideriamo il problema del Wumpus World:

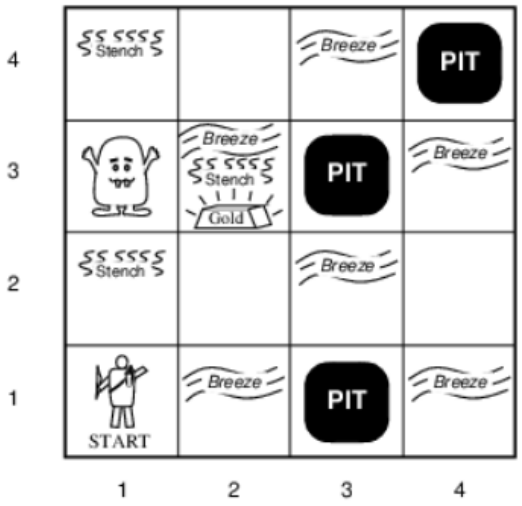



Figura 23: Esempio di Wumpus World

- **Performance measure**:
  - +1000 per uscire con l'oro
  - -1 per ogni azione
  - -1000 per morire
  - -10 per ogni freccia usata
- **Environment**:
  - 4x4 griglia

  - Wumpus in una casella
  - Pozzi in alcune caselle
  - Oro in una casella
  - Caselle adiacenti al Wumpus hanno puzza
  - Caselle adiacenti ai pozzi hanno brezza
  - Glitter nella casella con l'oro
  - Prendi per prendere l'oro dalla casella
  - Rilascia per lasciare l'oro nella casella
  - Sparare uccide il Wumpus se è nella stessa direzione della freccia
  - Sparare usa l'unica freccia disponibile
- **Actuators:**
  - Gira a sinistra
  - Gira a destra
  - Avanti
  - Prendi
  - Rilascia
- **Sensors:**
  - Puzza
  - Brezza
  - Glitter

Questo problema è:

- **Non osservabile** perchè si ha solo percezione locale
- **Deterministico** perchè le azioni hanno effetti certi
- **Non episodico** perchè le azioni sono sequenziali
- **Statico** perchè l'ambiente non cambia
- **Discreto** perchè ci sono un numero finito di stati e azioni
- **Singolo agente** perchè c'è solo un agente che agisce nell'ambiente (il wumpus è parte dell'ambiente)

## 4.2 Logica in generale

La logica è un sistema formale per rappresentare informazioni e ragionare su di esse.

- **Sintassi**: definisce le regole per costruire frasi valide nel linguaggio
- **Semantica**: definisce il significato delle frasi nel linguaggio, ad esempioo definisce quando una frase è vera o falsa in un certo mondo

### 4.2.1 Entailment (Derivazione logica)

La derivazione o entailment indica che da una certa cosa ne segue un altra:

$$\mathrm{KB} \models \alpha$$

La knowledge base implica una frase $\alpha$ se e solo se $\alpha$ è vera in tutti i mondi in cui è vera la knowledge base. La derivazione è una relazione tra frasi logiche (sintassi) che si basa sulla semantica. Un modello è un mondo formalmente strutturato rispetto a quale verità o falsità di frasi logiche può essere valutata. $\mathfrak{m}$ è un modello di una frase $\alpha$, se $\alpha$ è vera in $\mathfrak{m}$, allora $\mathrm{M}(\alpha)$ è l'insieme di tutti i modelli di $\alpha$. Di conseguenza

$$\mathrm{KB} \models \alpha \iff \mathrm{M}(\mathrm{KB}) \subseteq \mathrm{M}(\alpha)$$

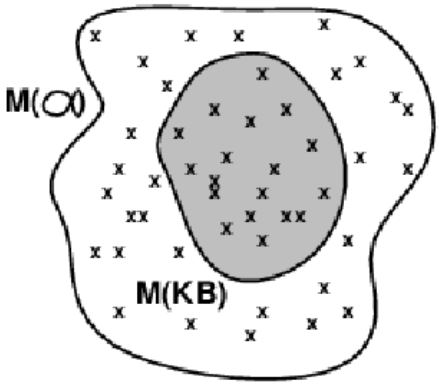



Figura 24: Esempio di entailment

**Esempio 4.2.** Un esempio potrebbe essere:

- KB = Juventus ha vinto e la Roma ha vinto
- $\alpha$ = Juventus ha vinto

### 4.2.2 Inferenza

L'inferenza è il processo di derivare nuove frasi logiche da frasi esistenti nella knowledge base utilizzando regole logiche. Una frase $\alpha$ può essere derivata dalla knowledge base KB con una procedura $\mathfrak{i}$:

$$\mathrm{KB} \vdash_{\mathfrak{i}} \alpha$$

Un sistema di inferenza ha due proprietà importanti:

- **Soundness**: se $\mathrm{KB} \vdash_{\mathfrak{i}} \alpha$ allora $\mathrm{KB} \models \alpha$ cioè tutto ciò che viene derivato è vero
- **Completeness**: se $\mathrm{KB} \models \alpha$ allora $\mathrm{KB} \vdash_{\mathfrak{i}} \alpha$ cioè tutto ciò che è vero può essere derivato

**Esempio 4.3.** Rappresentiamo il problema del Wumpus World in logica proposizionale:

- $P_{i,j}$: è vero se c'è un pozzo nella casella $(i, j)$
- $B_{i,j}$: è vero se c'è brezza nella casella $(i, j)$

Consideriamo i seguenti fatti:

$$R_1 : \neg P_{1,1}$$
$$R_2 : \neg B_{1,1}$$
$$R_3 : B_{2,1}$$

Rappresentiamo in forma logica la frase "i pozzi causano brezza nelle caselle adiacenti":

- Considerando caselle specifiche:

$$R_4 : B_{1,1} \iff (P_{1,2} \vee P_{2,1})$$
$$R_5 : B_{2,1} \iff (P_{1,1} \vee P_{2,2} \vee P_{3,1})$$

- R rappresenta le regole del mondo
- P rappresenta le variabili da assegnare
- KB è la knowledge base, cioè l'and logico di tutte le regole

### 4.2.3 Inferenza mediante enumerazione

Esiste un algoritmo che verifica per ogni possibile assegnamento se $KB \models \alpha$:

```
function TT-Entails?(KB, alpha) returns true or false
  inputs: KB, the knowledge base, a sentence in prop. logic
          alpha, the query, a sentence in prop. logic
  symbols <- a list of the proposition symbols in KB and alpha
  return TT-Check-All(KB, alpha, symbols, [ ])

function TT-Check-All(KB, alpha, symbols, model) returns true or
    false
  if Empty?(symbols) then
    if PL-True?(KB, model) then return PL-True?(alpha, model)
    else return true
  else do
    P <- First(symbols); rest <- Rest(symbols)
    return TT-Check-All(KB, alpha, rest, Extend(P, true, model))
    and
           TT-Check-All(KB, alpha, rest, Extend(P, false, model))
```

Siccome si devono provare tutte le possibili combinazioni di verità per $n$ variabili, la complessità è $O(2^n)$, quindi il problema è co-NP-completo.

### 4.2.4 Metodi di dimostrazione

I metodi di dimostrazione si dividono in due categorie principali:

- **Model checking**:

- Truth table enumeration: verifica tutti i modelli possibili
- Improved backtracking: usa backtracking per evitare di esplorare modelli non validi. Ad esempio DPLL (Davis-Putnam-Logemann-Loveland)
- Heuristic search in model space: cerca modelli validi usando euristiche per guidare la ricerca (corretto ma non completo). Ad esempio algoritmo come hill-climbing

• **Applicazione delle regole di inferenza**

- Generazione di nuove frasi logiche a partire da quelle esistenti
- Dimostrazione: Una sequenza di applicazioni di regole di inferenza
- Richiedono la traduzione di frasi logiche in **forma normale**

### 4.2.5 Equivalenza logica

Due frasi logiche sono logicamente equivalenti se e solo se sono vere negli stessi modelli:

$$\alpha \equiv \beta \iff \alpha \models \beta \land \beta \models \alpha$$

**Esempio 4.4.** Delle tautologie importanti sono:

$$\begin{aligned}
(\alpha \land \beta) &\equiv (\beta \land \alpha) \\
(\alpha \lor \beta) &\equiv (\beta \lor \alpha) \\
((\alpha \land \beta) \land \gamma) &\equiv (\alpha \land (\beta \land \gamma)) \\
((\alpha \lor \beta) \lor \gamma) &\equiv (\alpha \lor (\beta \lor \gamma)) \\
\neg(\neg\alpha) &\equiv \alpha \\
(\alpha \implies \beta) &\equiv (\neg\alpha \implies \neg\beta) \\
(\alpha \implies \beta) &\equiv (\neg\alpha \lor \beta) \\
(\alpha \iff \beta) &\equiv ((\alpha \implies \beta) \land (\beta \implies \alpha)) \\
\neg(\alpha \land \beta) &\equiv (\neg\alpha \lor \neg\beta) \\
\neg(\alpha \lor \beta) &\equiv (\neg\alpha \land \neg\beta) \\
(\alpha \land (\beta \lor \gamma)) &\equiv ((\alpha \land \beta) \lor (\alpha \land \gamma)) \\
(\alpha \lor (\beta \land \gamma)) &\equiv ((\alpha \lor \beta) \land (\alpha \lor \gamma))
\end{aligned} \tag{1}$$

### 4.2.6 Validità e soddisfacibilità

Una frase logica è **valida** se è vera in tutti i modelli, ad esempio:

$$\mathrm{True}, \quad \alpha \lor \neg\alpha, \quad \alpha \implies \alpha$$

La validità è collegata alla derivazione dal teorema della deduzione:

$$\mathrm{KB} \models \alpha \iff (\mathrm{KB} \implies \alpha) \text{ è valida}$$

Una frase logica è **soddisfacibile** se è vera in almeno un modello, ad esempio:

$$A \vee B, \quad C$$

Invece è **insoddisfacibile** se non è vera in nessun modello, ad esempio:

$$False, \quad A \wedge \neg A$$

La soddisfacibilità è collegata alla derivazione dal seguente teorema:

$$KB \models \alpha \iff (KB \wedge \neg\alpha) \text{ è insoddisfacibile}$$

## 4.3 Sistema di inferenza

Un sistema di inferenza è un insieme di regole che permettono di derivare nuove frasi logiche da frasi esistenti. Le regole sono scritte nella forma:

$$\frac{A_1 \dots A_k}{A} \qquad \frac{\text{Premesse}}{\text{Conclusioni}}$$

**Definizione 4.1.** Una derivazione $A$ è derivata da un insieme di formule $\Gamma$ con un sistema di inferenza $\mathcal{R}$ ($\Gamma \vdash_{\mathcal{R}} A$) se esiste una sequenza $A_1, \dots, A_n$ di formule tale che:

- $A_n = A$
- $\forall i \in \{1, \dots, n\}$ è vera una delle seguenti:
  1. $A_i \in \Gamma$
  2. $A_i$ è una derivazione diretta delle formule nella sequenza precedente

La sequenza $A_1, \dots, A_n$ è una **dimostrazione** di $A$. $\Gamma$ sono le **premesse** (assunzioni o ipotesi) per $A$.

### 4.3.1 Proprietà di un sistema di inferenza

- **Correttezza delle regole di inferenza**: Le conclusioni devono essere delle conseguenze logiche delle premesse
- **Completezza**: Se una formula è una conseguenza logica delle premesse, allora deve essere possibile derivarla usando le regole di inferenza
- **Completezza refutazionale**: Esiste una derivazione di $\square$ se le ipotesi unite alla negazione della conclusione sono insoddisfacibili:

$$\square \text{ è derivabile da } H \cup \{\neg\psi\} \iff \text{ è insoddisfacente}$$

## 4.4 Problema di deduzione

Un problema di deduzione consiste nel determinare se una certa formula logica può essere derivata da un insieme di formule esistenti:

$$\Gamma \models \alpha$$

Per risolverlo si possono usare due approcci principali:

- **Dimostrazione per assurdo** (reductio ad absurdum): Si dimostra che

  $$\Gamma \wedge \neg\alpha \text{ è insoddisfacibile}$$

  I principali metodi sono la **resolution**

- **Forware/backward reasoning**: È un algoritmo polinomiale corretto e completo per un insieme limitato di formule logiche (Horn clauses)

### 4.4.1 Resolution

Per usare la resolution si deve prima convertire ogni formula logica in **forma normale congiuntiva** (CNF). La CNF è una congiunzione di clausole (letterali), ad esempio:

$$(A \vee \neg B) \wedge (B \vee \neg C \vee \neg D)$$

La regola di inferenza della resolution è:

$$\frac{l_1 \vee \ldots \vee l_k \quad m_1 \vee \ldots \vee m_n}{l_1 \vee \ldots \vee l_{i-1} \vee l_{i+1} \vee \ldots \vee l_k \vee m_1 \vee \ldots \vee m_{j-1} \vee m_{j+1} \vee \ldots \vee m_n}$$

Dove $l_i$ e $m_j$ sono letterali complementari, cioè uno è la negazione dell'altro. La risoluzione è corretta e completa per la logica proposizionale.

**Esempio 4.5.** Alcune applicazioni della risoluzione sono le seguenti:

$$\frac{A \vee B \quad \neg B}{A}$$

### 4.4.2 Conversione in CNF

Consideriamo la formula logica:

$$B_{1,1} \iff (P_{1,2} \vee P_{2,1})$$

Per convertire una formula logica in CNF si seguono i seguenti passi:

1. Elimina $\iff$, sostituendo $\alpha \iff \beta$ con $(\alpha \implies \beta) \wedge (\beta \implies \alpha)$

   $$(B_{1,1} \implies (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \implies B_{1,1})$$

2. Elimina $\implies$, rimpiazzando $\alpha \implies \beta$ con $\neg\alpha \vee \beta$

   $$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg (P_{1,2} \vee P_{2,1}) \vee B_{1,1})$$

3. Muovi $\neg$ dentro usando le leggi di De Morgan e doppia negazione:

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge ((\neg P_{1,2} \wedge \neg P_{2,1}) \vee B_{1,1})$$

4. Applica la regola della distribuzione:

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})$$

L'algoritmo della risoluzione è il seguente:

```
function PL-Resolution(KB, alpha) returns true or false
  inputs: KB, the knowledge base, a sentence in propositional logic
          alpha, the query, a sentence in propositional logic
  clauses <- the set of clauses in the CNF representation of KB and
     not alpha
  new <- { }
  loop do
    for each Ci , Cj in clauses do
      resolvents <- PL-Resolve(Ci , Cj )
      if resolvents contains the empty clause then return true
      new <- new union resolvents
    if new is included in clauses then return false
    clauses <- clauses union new
```

La risoluzione è da applicare solo a due singoli letterali alla volta.

**Esempio 4.6.** Consideriamo la knowledge base:

$$\text{KB} = (B_{1,1} \iff (P_{1,2} \vee P_{2,1}) \wedge \neg B_{1,1}) \quad \alpha = \neg P_{1,2}$$

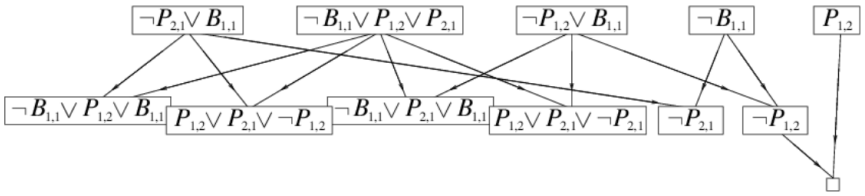



Figura 25: Esempio di risoluzione

## 4.5 Forward e backward chaining

Per usare il forward e backward chaining si deve prima convertire ogni formula logica in **Horn clauses**. Ciascuna clausola deve rispettare una delle seguenti regole:

- È un singolo simbolo proposizionale, oppure
- È una congiunzione di simboli proposizionali che implica un singolo simbolo proposizionale, ad esempio:

$$(A \wedge B) \implies C$$

Questa forma permette di avere come regola il **modus ponens**, cioè:

$$\frac{\alpha_1, \ldots, \alpha_n \quad \alpha_1 \wedge \ldots \wedge \alpha_n \implies \beta}{\beta}$$

Questi algoritmi hanno entrambi complessità lineare.

### 4.5.1 Forward chaining

L'idea di questo algoritmo è quella di trovare qualsiasi regola le cui premesse siano soddisfatte dalla knowledge base e aggiungere la conclusione alla knowledge base finchè non si trova la formula desiderata.

```
function PL-FC-Entails?(KB, q) returns true or false
  inputs: KB # knowledge base, a set of propositional Horn clauses
          q # query, a prop. symbol
  local variables: count # a table, indexed by clause, initially
    the number of premises
                   inferred # a table, indexed by symbol, each
    entry initially false
                   agenda # a list of symbols, initially the
    symbols known in KB
  while agenda is not empty do
    p <- Pop(agenda)
    unless inferred[p] do
      inferred[p] <- true
      for each Horn Clause c in whose premise p appears do
        decrement count[c]
        if count[c] = 0 then do
          if Head[c] = q then return true
          Push(Head[c], agenda)
  return false
```

**Definizione 4.2** (Dimostrazione di completezz)**.** Forward checking deriva tutte le formule atomiche che possono essere derivate dalla knowledge base.

1. Raggiunge un **punto fisso** dove non si possono più derivare nuove formule atomiche
2. Si considera lo stato finale come un modello $m$ assegnando true o false ai simboli
3. Ogni clausola nella knowledge base originale è vera in $m$
4. Quindi $m$ è un modello di KB
5. Per ogni $a$ (formula atomica), se la knowledge base deriva $a$:

   $$KB \models a$$

   allora $a$ è vera in **ogni** modello di KB, incluso $m$

### 4.5.2 Backward chaining

L'idea di questo algoritmo è quella di iniziare dalla formula desiderata e cercare di dimostrare che è vera cercando regole che la concludono e poi cercando di dimostrare che le premesse di queste regole sono vere. Bisogna fare attenzione a:

- Evitare cicli: controllare se il nuovo subgoal è già nella lista dei subgoal
- Evitare ripetizioni: evitare di dimostrare lo stesso subgoal più volte

### 4.5.3 Differenze tra forward e backward chaining

- **Forward chaining**: È **data-driven**, cioè un processo automatico che parte dai fatti noti
- **Backward chaining**: È **goal-driven**, cioè parte da un obiettivo specifico da raggiungere. La complessità può essere molto meglio di $\mathrm{O(n)}$ nella dimensione della knowledge base

# 5 Rappresentare l'incertezza

**Esempio 5.1.** Consideriamo l'azione $A_t$ = lasciare l'aereoporto t minuti prima del volo.

Ci chiediamo se $A_t$ ci porterà al volo in orario. I problemi sono:

- **Osservabilità parziale**: non sappiamo se ci saranno ingorghi stradali, condizioni meteo avverse, ecc...
- **Sensori rumorosi**: le previsioni del tempo e le condizioni del traffico non sono sempre accurate
- **Incertezza nel risultato delle azioni**: si potrebbero avere imprevisti come incidenti stradali, guasti dell'auto, ecc...
- **Complessità immensa**: non è possibile modellare e predirre ogni possibile evento

Ci sono talmente tante precondizioni che non possiamo tenerle tutte in considerazione.

Ci sono tre principali metodi per gestire l'incertezza:

- **Default o nonmonotonic logic**: si fanno assunzioni che sono vere a meno che non si dimostri il contrario
- **Fuzzy logic**: cerca di rendere la logica non binaria, permettendo di esprimere **gradi di verità**, ad esempio valori tra 0 e 1
- **Probabilità**: gli eventi sono binari (veri o falsi), ma con una certa probabilità.

## 5.1 Probabilità

Le asserzioni probabilistiche sommano gli effetti di:

- **Lazyness**: mancanza di rappresentare informazioni complete
- **Ignorance**: mancanza di conoscenza su fatti specifici

Esiste anche la probabilità **soggettiva** o **Bayesiana**, che rappresenta il grado di fiducia in una certa asserzione, basata sulle conoscenze attuali, ad esempio:

$$P(A_25 \mid \text{ nessun incidente riportato}) = 0.06$$

Questi non sono gradi di verità, ma gradi di **conoscenza** (belief). Le probabilità cambiano in base alle nuove evidenze.

### 5.1.1 Decidere con incertezza

Consideriamo di avere le seguenti probabilità:

$$\begin{aligned} P(A_25 \text{ mi fa arrivare in tempo} \mid \ldots) &= 0.04 \\ P(A_90 \text{ mi fa arrivare in tempo} \mid \ldots) &= 0.70 \\ P(A_120 \text{ mi fa arrivare in tempo} \mid \ldots) &= 0.95 \\ P(A_1440 \text{ mi fa arrivare in tempo} \mid \ldots) &= 0.9999 \end{aligned}$$

Non è immediato quale azione scegliere, questo dipende sulle **preferenze** di chi sceglie. Le preferenze sono rappresentate dalla **Utility theory**. Per scegliere l'azione che massimizza l'utilità attesa si usa la **Decision theory** che combina l'utility theory con la **Probability theory Maximum Expected Utility (MEU)**.

### 5.1.2 Basi di probabilità

- Il **sample space** $\Omega$ è l'insieme di tutti i possibili risultati di un esperimento casuale
- $\omega \in \Omega$ è un **sample point** (o mondo possibile, o evento atomico)
- Un **modello probabilistico** è un **sample space** con un assegnamento di probabilità $P(\omega)$ per ogni $\omega \in \Omega$
- Un evento $A$ è un sottoinsieme di $\Omega$. La probabilità di un evento è la somma delle probabilità dei sample point che lo compongono:

$$P(A) = \sum_{\omega \in A} P(\omega)$$

- Le variabili casuali possono avere vari domini e sono soggette al cambiamento, cioè non si possono decidere a priori i loro valori. $P$ induce una **distribuzione di probabilità** su ogni variabile casuale, cioè una funzione che assegna una probabilità ad ogni possibile valore della variabile:

$$P(X = x_i) = \sum_{\omega \in \Omega \mid X(\omega) = x_i} P(\omega)$$

- Una **proposizione** è la probabilità che una variabile casuale assuma un certo valore:

  $P(X = x_i)$ è la proposizione "X assume il valore $x_i$"

  Le proposizioni possono essere usate anche per creare modelli di logica proposizionale, ad esempio date due variabili casuali $A$ e $B$:

- evento $\mathrm{a}$ = insieme di sample points in cui $\mathrm{A}(\omega) = \mathit{true}$
- evento $\neg \mathrm{a}$ = insieme di sample points in cui $\mathrm{A}(\omega) = \mathit{false}$
- evento $\mathrm{a} \wedge \mathrm{b}$ = punti dove $\mathrm{A}(\omega) = \mathit{true}$ e $\mathrm{B}(\omega) = \mathit{true}$

La probabilità implica che due eventi in relazione tra loro hanno probabilità collegate. Quando si calcolano le probabilità quindi bisogna considerare l'intersezione degli eventi, sottraendo le sovrapposizioni dalla somma delle probabilità.

### 5.1.3 Variabili casuali

Le variabili casuali sono proposizioni atomiche, mentre invece le proposizioni composte sono create combinando variabili casuali. I tipi di variabili casuali sono:

- **Proposizionali o booleane**: assumono valori vero o falso, ad esempio:

  $$\text{Carie} \;=\; \text{ho una carie?}$$

  Carie $= \mathit{true}$ è una proposizione atomica, scritta anche come $\mathit{carie}$

- **Discrete** (finite o infinite): assumono un insieme numerabile di valori

  $$\text{Previsioni del tempo} \;= \langle \text{sole, pioggia, neve, nuvoloso} \rangle$$

  Previsioni del tempo $=$ pioggia è una proposizione. I valori devono essere esaustivi e mutuamente esclusivi

- **Continue** (limitate o illimitate): assumono un insieme non numerabile di valori

  $$\text{Temp} \;= 21.6$$

### 5.1.4 Eventi atomici

Gli eventi atomici sono assegnazioni di tutte le variabili casuali. Gli eventi atomici hanno le seguenti proprietà:

1. Mutuamente esclusivi, cioè non possono essere veri contemporaneamente
2. Esaustivi, cioè uno di essi deve essere vero
3. Consegue una verità per ogni proposizione
4. Qualsiasi proposizione è logicamente equivalente ad una disgiunzione di eventi atomici rilevanti

Gli eventi atomici sono come i modelli per la logica proposizionale.

### 5.1.5 Probabilità a priori

La probabilità a priori è la probabilità di un evento senza alcuna conoscenza aggiuntiva. È analoga ai fatti nella knowledge base logica.

### 5.1.6 Probabilità congiunta

La probabilità congiunta rappresenta la probabilità di ogni evento atomico di un insieme di variabili casuali.

**Esempio 5.2.** Consideriamo la probabilità:

$$P(\text{Weather, Cavity})$$

Questa probabilità condizionata è una matrice di valori:

| **Weather =** | Sunny | Rain | Cloudy | Snow |
|---|---|---|---|---|
| Cavity = True | 0.144 | 0.02 | 0.016 | 0.02 |
| Cavity = False | 0.576 | 0.08 | 0.064 | 0.08 |

Tabella 2: Esempio di probabilità condizionata

Se si sommano tutte queste probabilità si ottiene 1, mentre se si sommano le probabilità per una riga o colonna si ottiene la probabilità marginale, cioè la probabilità incondizionata di una variabile casuale.

### 5.1.7 Probabilità continua

La probabilità continua è definita tramite una funzione parametrica di un valore chiamata **funzione di densità di probabilità** (PDF).

### 5.1.8 Probabilità condizionata

La probabilità condizionata rappresenta la probabilità di un evento dato che un altro evento sia vero. Ad esempio:

$$P(\text{carie} \mid \text{dolore}) = 0.6$$

rappresenta che la probabilità di avere una carie, dato che l'unica informazione disponibile è quella in cui si ha dolore, è del 60%.

La notazione per distribuzioni condizionate è:

$$P(C \mid T)$$

che è un vettore da due vettori di due elementi:

$$P(C \mid T) = [[P(c \mid |t), P(\neg c \mid t)], [P(c \mid \neg t), P(\neg c \mid \neg t)]]$$

Alcune informazioni possono essere irrilevanti e quindi possono essere ignorate. La probabilità condizionata può essere calcolata come:

$$P(a \mid b) = \frac{P(a \wedge b)}{P(b)} \text{ se } P(b) > 0$$

dove $P(a \wedge b)$ è la probabilità congiunta di $a$ e $b$ e può essere anche scritta come (regola del prodotto):

$$P(a \wedge b) = P(a \mid b) \cdot P(b) = P(b \mid a) \cdot P(a)$$

Esiste anche la regola della catena che permette di rappresentare una probabilità generica utilizzando i condizionali:

$$\begin{aligned} P(X_1, \ldots, X_n) &= P(X_1, \ldots, X_{n-1}) \cdot P(X_n \mid X_1, \ldots, X_{n-1}) \\ &= P(X_1, \ldots, X_{n-2}) P(X_{n-1} \mid X_1, \ldots, X_{n-2}) P(X_n \mid X_1, \ldots, X_{n-1}) \\ &= \ldots \\ &= \prod_{i=1}^{n} P(X_i \mid X_1, \ldots, X_{i-1}) \end{aligned}$$

### 5.1.9 Inferenza per enumerazione

Per ogni proposizione $\varphi$, si sommano gli eventi atomici dove è vero:

$$P(\varphi) = \sum_{\omega : \omega \models \varphi} P(\omega)$$

Ricordando che:

- Ogni proposizione $\varphi$ è equivalente alla disgiunzione degli eventi atomici in cui è vera
- Gli eventi atomici sono mutuamente esclusivi

**Esempio 5.3.** Consideriamo la seguente tabella di probabilità congiunta:

| | *toothache* | | ¬ *toothache* | |
|---|---|---|---|---|
| | *catch* | ¬ *catch* | *catch* | ¬ *catch* |
| *cavity* | .108 | .012 | .072 | .008 |
| ¬ *cavity* | .016 | .064 | .144 | .576 |

Figura 26: Tabella di esempio

La probabilità di avere *toothache* è data dalla somma delle probabilità degli eventi atomici in cui *toothache* è vero:

| | *toothache* | | ¬ *toothache* | |
|---|---|---|---|---|
| | *catch* | ¬ *catch* | *catch* | ¬ *catch* |
| *cavity* | .108 | .012 | .072 | .008 |
| ¬ *cavity* | .016 | .064 | .144 | .576 |

Figura 27: Calcolo della probabilità di toothache

$$P(\mathrm{toothache}) = 0.108 + 0.012 + 0.016 + 0.064 = 0.2$$

Si può anche calcolare la probabilità congiunta di due eventi:

| | *toothache* | | ¬ *toothache* | |
|---|---|---|---|---|
| | *catch* | ¬ *catch* | *catch* | ¬ *catch* |
| *cavity* | .108 | .012 | .072 | .008 |
| ¬ *cavity* | .016 | .064 | .144 | .576 |

Figura 28: Calcolo della probabilità congiunta di toothache e cavity

$$\mathrm{P}(\mathrm{cavity} \wedge \mathrm{toothache}) = 0.108 + 0.012 + 0.072 + 0.008 + 0.016 + 0.064 = 0.28$$

Oppure anche probabilità condizionate:

| | *toothache* | | ¬ *toothache* | |
|---|---|---|---|---|
| | *catch* | ¬ *catch* | *catch* | ¬ *catch* |
| *cavity* | .108 | .012 | .072 | .008 |
| ¬ *cavity* | .016 | .064 | .144 | .576 |

Figura 29: Calcolo della probabilità condizionata di not cavity dato toothache

$$\begin{aligned}\mathrm{P}(\neg \mathrm{cavity} \mid \mathrm{toothache}) &= \frac{\mathrm{P}(\neg \mathrm{cavity} \wedge \mathrm{toothache})}{\mathrm{P}(\mathrm{toothache})} \\ &= \frac{0.016 + 0.064}{0.108 + 0.012 + 0.016 + 0.064} \\ &= 0.4\end{aligned}$$

### 5.1.10 Normalizzazione

L'idea è quella di calcolare la distribuzione di probabilità su una variabile di interesse fissando delle **variabili di evidenza** e sommando su tutte le **variabili nascoste**, cioè quelle non di interesse.

**Esempio 5.4.** Consideriamo la tabella dell'esempio precedente, si può calcolare la probabilità condizionata e considerare il denominatore come una costante di normalizzazione:

| | *toothache* | | ¬ *toothache* | |
|---|---|---|---|---|
| | *catch* | ¬ *catch* | *catch* | ¬ *catch* |
| *cavity* | .108 | .012 | .072 | .008 |
| ¬ *cavity* | .016 | .064 | .144 | .576 |

Figura 30: Calcolo della probabilità condizionata di cavity dato toothache

$$
\begin{aligned}
\mathrm{P}(\mathrm{Cavity} \mid \mathrm{Toothache}) &= \alpha \mathrm{P}(\mathrm{Cavity}, \mathrm{Toothache}) \\
&= \alpha \begin{bmatrix} \mathrm{P}(\mathrm{Cavity} = \mathrm{true}, \mathrm{Toothache} = \mathrm{true}) \\ \mathrm{P}(\mathrm{Cavity} = \mathrm{false}, \mathrm{Toothache} = \mathrm{true}) \end{bmatrix} \\
&= \alpha \begin{bmatrix} \langle 0.108, 0.016 \rangle \\ \langle 0.012, 0.064 \rangle \end{bmatrix} \\
&= \alpha \langle 0.12, 0.08 \rangle \\
&= \langle 0.6, 0.4 \rangle
\end{aligned}
$$

Il risultato finale è un vettore la cui somma dei valori è 1.

## 5.2 Indipendenza

Due variabili casuali $A$ e $B$ sono indipendenti se e solo se:

$$\mathrm{P}(A \mid B) = \mathrm{P}(A) \quad \text{oppure} \quad \mathrm{P}(B \mid A) = \mathrm{P}(B) \text{ oppure} \quad \mathrm{P}(A, B) = \mathrm{P}(A) \cdot \mathrm{P}(B)$$

### 5.2.1 Indipendenza condizionata

Non sempre si possono considerare due variabili casuali come indipendenti, ma possono esistere delle terze variabili che non influenzano la relazione tra le prime due.

**Esempio 5.5.** Consideriamo l'esempio del dentista, se sappiamo che una persona ha una carie, la probabilità che la sonda del dentista si incastri tra i denti non dipende dal fatto che la persona abbia dolore ai denti o meno. Quindi:

$$\mathrm{P}(\mathrm{catch} \mid \mathrm{toothache}, \mathrm{cavity}) = \mathrm{P}(\mathrm{catch} \mid \mathrm{cavity})$$

e lo stesso vale se la persona non ha una carie:

$$\mathrm{P}(\mathrm{catch} \mid \mathrm{toothache}, \neg\mathrm{cavity}) = \mathrm{P}(\mathrm{catch} \mid \neg\mathrm{cavity})$$

Quindi $\mathrm{Catch}$ è **condizionatamente indipendente** da $\mathrm{Toothache}$ dato $\mathrm{Cavity}$:

$$\mathrm{P}(\mathrm{Catch} \mid \mathrm{Toothache}, \mathrm{Cavity}) = \mathrm{P}(\mathrm{Catch} \mid \mathrm{Cavity})$$

Si può quindi ignorare una variabile quando si calcola la probabilità condizionata.

### 5.2.2 Regola di Bayes

La regola di Bayes permette di calcolare la probabilità condizionata invertendo i condizionali:

$$P(causa \mid effetto) = \frac{P(effetto \mid causa) \cdot P(causa)}{P(effetto)}$$

Questa regola è utile quando $P(effetto \mid causa)$ è più facile da calcolare rispetto a $P(causa \mid effetto)$.

# 6 Sequential Decision Making

Le azioni vengono raramente scelte individualmente, ma sono parte di **sequenze** di azioni. Per valutare un'azione si deve andare oltre il semplice risultato immediato e considerare:

- Utilità a lungo termine che deriva dall'azione
- Acquisire nuove informazioni che possono influenzare le azioni future

**Esempio 6.1.** Consideriamo il problema di esplorazione di un labirinto in cui le azioni hanno una certa probabilità di successo:

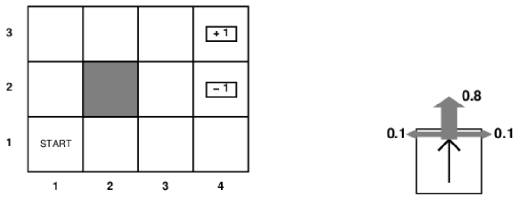



Figura 31: Esempio di labirinto

- Stati $s \in S$
- Azioni $a \in A$
- Modello: $T(s, a, s') \equiv P(s'|s, a)$ è la probabilità che l'azione $a$ nello stato $s$ porti allo stato $s'$
- Funzione di ricompensa: $R(s)$ (o $R(s, a), R(s, a, s')$)

$$R(s) = \begin{cases} -0.04 & \text{(piccola penalità) se lo stato non è terminale} \\ \pm 1 & \text{Per gli stati terminali} \end{cases}$$

Un approccio può essere il seguente:

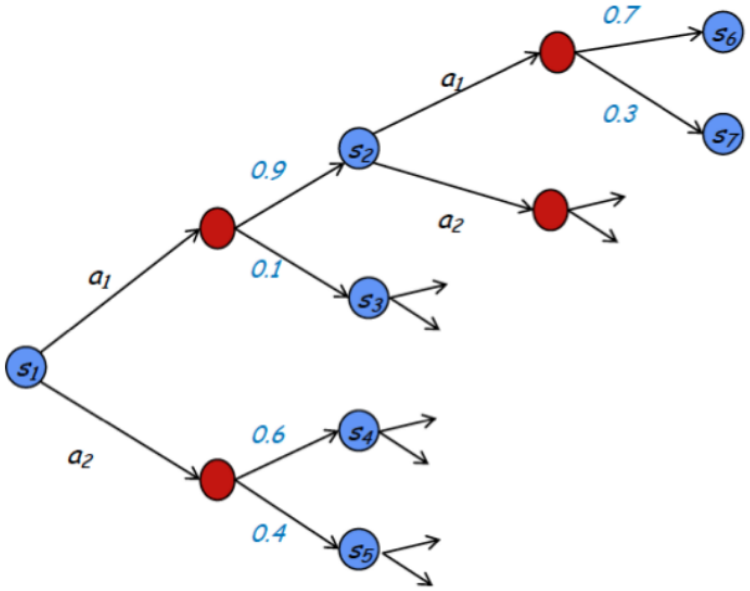



Figura 32: Esempio di approccio al labirinto

- I nodi rossi sono nodi chance, cioè rappresentano l'incertezza
- I nodi blu sono nodi di decisione, cioè rappresentano le azioni

I passi per calcolare la miglior sequenza di azioni sono:

1. Assegnare un utilità per ogni traiettoria, ad esempio:

$$u(s_1 \to s_2 \to s_6)$$

2. Per ogni sequenza di azioni calcola la probabilità di ogni traiettoria:

$$\Pr(s_1 \to s_2 \to s_6 \mid [a_1, a_1]) = 0.9 * 0.7 = 0.63$$

3. Calcola EU (Expected Utility) per ogni sequenza di azioni:

$$EU([a_1, a_1]), EU([a_1, a_2]), \ldots$$

4. Scegli la sequenza di azioni con la massima EU

I problemi di questo approccio sono:

- **Concettuale**: valutare tutte le sequenze delle azioni **senza considerare l'outcome reale non è la miglior strategia**
- **Pratico**: l'utilità per una sequenza è tipicamente più difficile da stimare rispetto all'utilità di singoli stati

- **Computazionale**: il numero di traiettorie cresce esponenzialmente

$$k^t n^t$$

dove:

* $k$ = numero di azioni
* $n$ = outcome per ogni azione
* $t$ = lunghezza della sequenza

Nei problemi di ricerca l'obiettivo è quello di trovare una **sequenza di azioni ottimale**. Considerando l'incertezza, l'obiettivo diventa quello di trovare una **politica ottimale** (policy) $\pi(s)$, ad esempio l'azione migliore per ogni possibile stato. La politica ottimale massimizza **la somma delle ricompense attese**.

## 6.1 Decision trees

Un albero di decisione è una rappresentazione ad albero delle decisioni che l'agente deve fare.

**Esempio 6.2.** Consideriamo il seguente albero di decisione non deterministico:

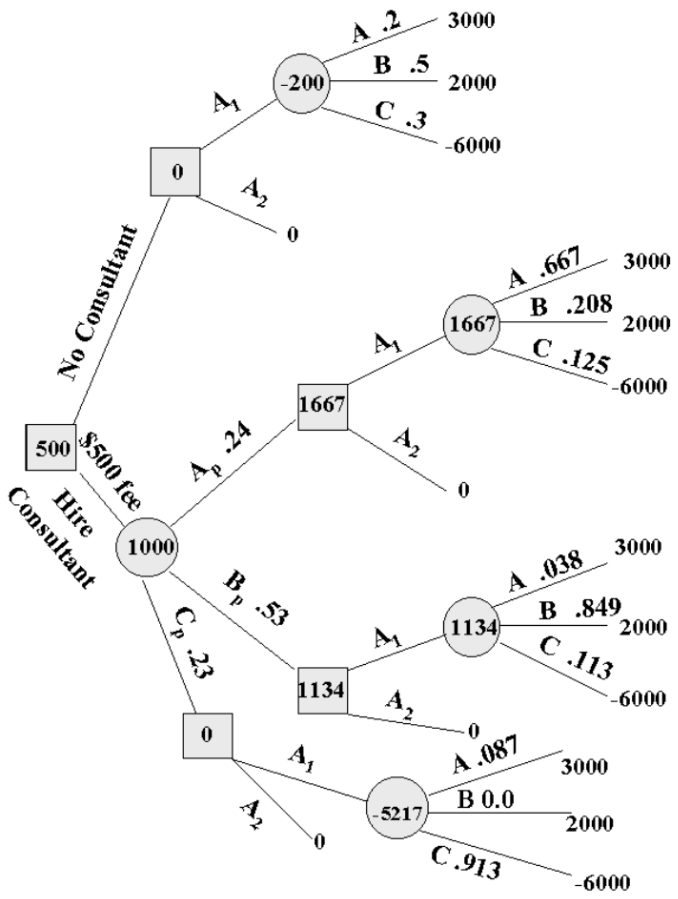



Figura 33: Esempio di albero di decisione

In questo albero:

- I nodi quadrati rappresentano le decisioni dell'agente. Il valore rappresenta il massimo tra le expected utility dei nodi figli
- I nodi rotondi rappresentano i nodi chance. Il valore del nodo è l'expected utility, ad esempio:

$$0.2 \cdot 3000 + 0.5 \cdot 2000 + 0.3 \cdot (-6000) = -200$$

### 6.1.1 Risoluzione di alberi di decisione

Esiste un algoritmo che risolve gli alberi di decisione chiamato **backward induction** (o rollback o expectimax). L'idea dell'algoritmo è:

1. Si parte dalle radici e si usa la maximum expected utility
2. Il valore di un nodo foglia C è:

$$\mathrm{EU}(C) = V(C)$$

3. Il valore di un nodo chance C non foglia è:

$$EU(C) = \sum_{D \in Child(C)} Pr(D) \cdot EU(D)$$

4. Il valore di un nodo decisione D è:

$$EU(D) = \max_{C \in Child(D)} EU(C)$$

5. La policy massimizza l'utility dei nodi decisione:

$$\pi(D) = \arg\max_{C \in Child(D)} EU(C)$$

Un decision tree con nodi ripetuti si espande in un albero molto grande.

## 6.2 Markov Decision Processes

Un Markov Decision Process (MDP) è una classe generale di problemi di decisione sequenziali più efficiente dei decision tree. Un MDP è definito da una tupla di 4 componenti:

$$\langle S, A, R, Pr \rangle$$

dove:

- $S$: insieme finito di stati $|S| = n$
- $A$: insieme finito di azioni $|A| = m$
- $p(s' \mid s, a) = Pr\{S_{t+1} = s' \mid S_t = s, A_t = a\}$: Funzione di transizione
- $r(s', a, s) = \mathbb{E}[R_{t+1} \mid S_{t+1} = s', A_t = a, S_t = s]$: Funzione di ricompensa

Questa rappresentazione deriva dalla **Markov Chain**, cioè che dato uno stato corrente il futuro è indipendente dal passato. Nelle MDP le azioni e gli stati passati sono irrilevanti quando si prende una decisione in un certo stato.

### 6.2.1 Proprietà

Le proprietà delle MDP sono:

- **Markov Dynamics**: Indipendenza dalla storia:

  $$Pr\{R_{t+1}, S_{t+1} \mid S_0, A_0, R_1, \ldots, S_{t-1}, A_{t-1}, R_t, S_t, A_t\}$$

  Se ci si trova in uno stato, le azioni e le ricompense passate non influenzano la probabilità del prossimo stato e della prossima ricompensa

- **Stazionarietà**: Le dinamiche non cambiano nel tempo:

  $$Pr\{R_{t+1}, S_{t+1} \mid S_t, A_t\} = Pr\{R_{t'+1}, S_{t'+1} \mid S_{t'}, A_{t'}\} \forall t, t'$$

  Cioè le probabilità di transizione e le ricompense non dipendono dal tempo

- **Completa osservabilità**: Non si può predirre esattamente che stato si raggiungerà, però si conosce sempre lo stato corrente

**Esempio 6.3.** Consideriamo l'esempio di un robot che deve muoversi in un ambiente e riciclare delle lattine. Le possibili azioni sono:

- Cerca una lattina (alta probabilità, ma potrebbe finire la batteria)
- Aspettare che qualcuno gli porti una lattina (bassa probabilità, ma non consuma batteria)
- Andare alla stazione di ricarica per caricare la batteria

L'agente decide in base al livello di batteria: $\{\mathrm{low}, \mathrm{high}\}$. Le azioni dipendono dallo stato:

$$A(\mathrm{low}) = \{search, wait\}$$
$$A(\mathrm{high}) = \{search, wait, recharge\}$$

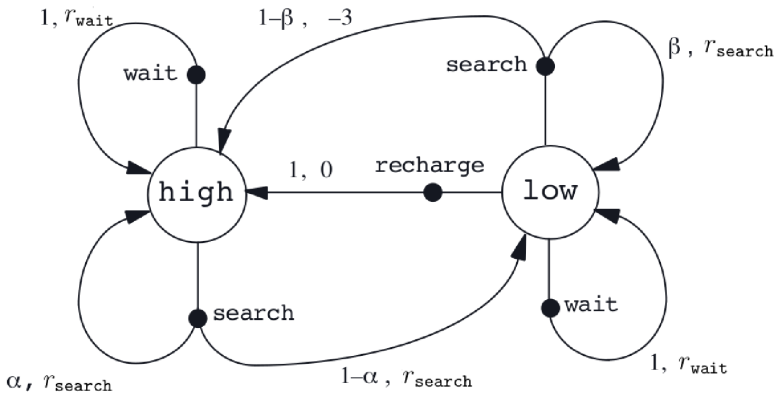



Figura 34: Esempio di MDP

Si vuole capire la scelta migliore che il robot deve fare in ogni stato per massimizzare la ricompensa attesa.

### 6.2.2 Tipi di policy

Esistono diversi tipi di policy:

- **Policy non stazionaria**: la politica dipende dal tempo t:

  $$\pi : S \times T \to A$$

  $\pi(s, t)$ è l'azione allo stato $s$ con $t$ passi rimanenti
- **Policy stazionaria**: la politica non dipende dal tempo:

  $$\pi : S \to A$$

  $\pi(s)$ è l'azione allo stato $s$ (indipendente dal tempo)
- **Policy stocastica**: la politica assegna una distribuzione di probabilità sulle azioni:
  $\pi(a \mid s)$ è la probabilità di scegliere l'azione $a$ nello stato $s$

### 6.2.3 Decisione tra policy

Bisogna trovare un metodo per valutare e confrontare le policy, cioè una sequenza di ricompense. Tipicamente si considerano le **stationary preferences**:

$$[r, r_0, r_1, r_2, \ldots] \succ [r, r'_0, r'_1, r'_2, \ldots] \iff [r_0, r_1, r_2, \ldots] \succ [r'_0, r'_1, r'_2, \ldots]$$

**Teorema 6.1.** Ci sono soltanto due modi per combinare le ricompense nel tempo:

1. **Funzione di utilità additiva**:

   $$U([s_0, s_1, s_2, \ldots]) = R(s_0) + R(s_1) + R(s_2) + \ldots$$

   Le ricompense future hanno lo stesso peso di quelle presenti.

2. **Funzione di utilità scontata**:

   $$U([s_0, s_1, s_2, \ldots]) = R(s_0) + \gamma R(s_1) + \gamma^2 R(s_2) + \ldots$$

   dove $0 < \gamma < 1$ è il fattore di sconto. Più la ricompensa è nel futuro, meno valore ha.

### 6.2.4 Valore di una policy

Bisogna trovare un valore per determinare quanto è buona una policy:

$$V : S \to \mathcal{R}$$

Questa funzione associa un valore considerando le **ricompense acccumulate**. Mentre invece la funzione $v_\pi(s)$ rappresenta il valore della policy $\pi$ nello stato $s$, cioè le ricompense accumulate attese in un orizzonte di tempo di interesse.

Se le sequenze di ricompense sono infinite, cioè si hanno infiniti stati (infinite horizon problems), si possono utilizzare le seguenti soluzioni:

- **Si sceglie un orizzonte finito**:
  - Si termina dopo un certo numero di passi T
  - Produce policy non stazionarie
- **Absorbing states**: garantiscono che per ogni policy verrà raggiunto eventualmente uno stato terminale
- **Discounting**: si usa un fattore di sconto $\gamma$ per ridurre l'importanza delle ricompense future:

  $$U([r_0, \ldots, r_\infty]) = \sum_{t=0}^{\infty} \gamma^t r_t \leqslant \frac{r_{max}}{1-\gamma} \quad \forall 0 < \gamma < 1$$

  Quindi la serie converge.
  Se si utilizza un $\gamma$ più basso si ragiona su un orizzonte più breve e quindi ricompense ottenute prima hanno più utilità rispetto a quelle ottenute nel futuro.

Quindi in breve:

- Se l'orizzonte T è finito si usa la ricompensa totale attesa data la policy $\pi$
- Se l'orizzonte è infinito si usa la somma delle ricompense accumulate attese scontate data la policy $\pi$
- Si può anche usare la ricompensa media per ogni passo

### 6.2.5 Risolvere un MDP

La soluzione di un MDP è una policy ottimale $\pi^*$ che massimizza l'expected utility da ogni stato $s$ seguendo quella policy $\pi$, cioè la ricompensa scontata accumulata attesa:

$$v_\pi(s) = \mathbb{E}\left\{\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \mid s_t = s\right\}$$

La Q-value (quality functiono action-value) è il vaore di prendere un'azione $a$ nello stato $s$ seguendo la policy $\pi$:

$$q_\pi(s,a) = \sum_{s'} p(s' \mid a, s)\,(r(s,a,s') + \gamma v_\pi(s'))$$

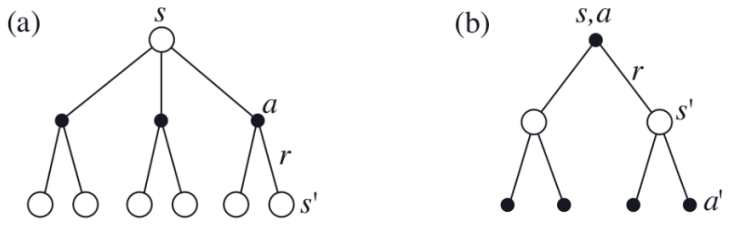
**Definizione utile 6.1.** Si nota che:

$$v_\pi(s) = q_\pi(s, \pi(s))$$

### 6.2.6 Equazioni di Bellman

Le equazioni di Bellman forniscono una relazione ricorsiva tra i valori degli stati e i valori delle azioni. I valori dello stato iniziale deve essere uguale al valore (scontato) atteso dello stato successivo più la ricompensa attesa lungo la transizione:

$$v_\pi(s) = \sum_{s'} p(s' \mid \pi(s), s)\,(r(s,\pi(s),s') + \gamma v_\pi(s'))$$




Back-up diagrams for $v_\pi$ and $q_\pi$

Figura 35: Rappresentazione delle equazioni di Bellman

Una policy è ottima se e solo se:

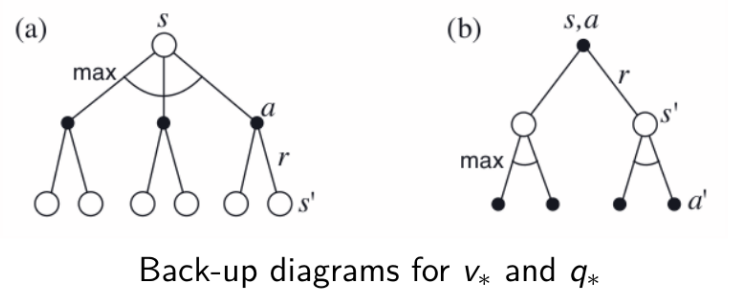
$$v_{\pi^*}(s) \geqslant v_\pi(s) \quad \forall s, \pi$$

Il valore dell'expected utility partendo da $s$ e agendo in modo ottimale dopo è:

$$v_*(s) = \max_\pi v_\pi(s)$$

Il valore ottimale del Q-value è:

$$q_*(s, a) = \max_\pi q_\pi(s, a)$$




Back-up diagrams for $v_*$ and $q_*$

Figura 36: Rappresentazione delle equazioni di Bellman per la policy ottimale

La policy ottima è calcolata come:

$$\pi^*(s) = \max_{a \in \mathcal{A}(s)} q_*(s, a) = \max_{a \in \mathcal{A}(s)} \sum_{s'} p(s' \mid a, s) \left(r(s, a, s') + \gamma v_*(s')\right)$$

### 6.2.7 Value iteration

Il value iteration è un algoritmo per calcolare la policy ottimale. L'idea è quella di trasformare l'equazione di ottimalità di Bellman in un'operazione di aggiornamento iterativo, combinando la valutazione della policy (calcolando il valore $v_\pi$ di una data policy) e il miglioramento della policy (rendendo la policy greedy rispetto al valore calcolato). La funzione di aggiornamento è:

$$v_{k+1}(s) = \max_a \sum_{s'} p(s' \mid a, s) \left(r(s, a, s') + \gamma v_k(s')\right)$$

L'algoritmo è il seguente:

```
Initialize array v with zeros for all s in S

do
  delta = 0
  for s in S:
      v_old = v[s]
      v[s] = max_a sum_{s'} p(s'|a,s) * (r(s,a,s') + gamma * v[s'])
      delta = max(delta, abs(v_old - v[s]))
while delta > threshold

Output a deterministic policy pi such that:
  pi(s) = argmax_a sum_{s'} p(s'|s,a) * (r(s,a,s') + gamma * v[s'])
```

Le caratteristiche dell'algoritmo sono:

- Garantisce la convergenza al valore ottimale $v_*$.
- Per orizzonti infiniti la policy ottima è stazionaria.
- La complessità per ogni iterazione è quadratica nel numero degli stati e lineare nel numero delle azioni: $O(n^2 m)$.
- Il rate di convergenza è lineare.

### 6.2.8 Policy iteration

La policy iteration è un altro algoritmo per calcolare la policy ottimale. L'idea è quella di alternare la valutazione della policy e il miglioramento della policy fino a quando la policy non cambia più. Quindi i passi dell'algoritmo sono:

```
pi = an arbitrary initial policy
repeat until no change in pi
  compute utilities given pi # policy evaluation
  update pi as if utilities are correct # policy improvement
```

1. **Policy evaluation**: per calcolare le utilities data una policy $\pi$ fissa, si risolve il seguente sistema di $n$ equazioni lineari ad $n$ incognite:

$$v_(s) = \sum_{s'} p(s' \mid \pi(s), s) \left(r(s, \pi(s), s') + \gamma v(s')\right)$$

   La risoluzione viene fatta in $O(n^3)$.

2. **Policy improvement**: dato il valore di tutti gli stati $v(s)$:
   - Cambiare in modo greedy la prima azione presa in uno stato basandosi sul valore corrente degli stati
   - Se il valore dello stato può essere migliorato, la nuova azione viene utilizzata dalla policy e quindi la performance della policy migliora

Questo algoritmo è molto costoso, quindi una modifica è quella di eseguire pochi step di value iteration (ma con $\pi$ fissa) iniziando dalla funzione di valore dell'iterazione

precedente e poi eseguire la policy iteration. L'algoritmo è il seguente:

1. Initialization
$v(s) \in \mathbb{R}$ and $\pi(s) \in A(s)$ arbitrarily $\forall s \in S$

2. Policy Evaluation
Repeat
  $\Delta \leftarrow 0$
  For each $s \in S$ :
    $v_{old} \leftarrow v(s)$
    $v(s) \leftarrow \sum_{s'} p(s' \mid s, \pi(s)) \left[ r(s, \pi(s), s') + \gamma v(s') \right]$
    $\Delta \leftarrow \max(\Delta, |v_{old} - v(s)|)$
Until $\Delta < \theta$ (small pos number)

3. Policy Improvement
policy_stable $\leftarrow true$
For each $s \in S$ :
  old_action $\leftarrow \pi(s)$
  $\pi(s) \leftarrow \arg\max_a \sum_{s'} p(s' \mid s, a) \left[ r(s, a, s') + \gamma v(s') \right]$
  If old_action $\neq \pi(s)$ then policy_stable $\leftarrow false$
If policy_stable
  Then return $\pi$ and $v$
  Else go to step 2.

Questo algoritmo garantisce l'ottimalità della policy.

## 6.3 MDP parzialmente osservabili

I POMDP (Partially Observable Markov Decision Processes) sono un modello di osservazione $O(s, e)$ che definisce la probabilità che l'agente ottiene l'evidenza $e$ quando si trova nello stato $s$. L'agente non conosce lo stato corrente.

**Teorema 6.2.** La policy ottima in un POMDP è una funzione $\pi(b)$ dove $b$ è il **belief state**, cioè una distribuzione di probabilità sugli stati possibili in cui l'agente può trovarsi.

Si può convertire un POMDP in un MDP nello spazio dei belief state, dove $T(b, a, b')$ è la probabilità che il nuovo belief state sia $b'$ sapendo che il belief state corrente è $b$ e l'azione eseguita è $a$.

# 7 Machine learning

Il machine learning consiste nel creare modelli partendo da un insieme di dati. Ci sono diversi tipi di learning:

- Supervised learning
- Unsipervised learning
- Reinforcement learning

## 7.1 Concetti di base e terminologia

- **Variabili di input**:
  - Variabili indipendenti o predittori o features
  - $X$, o quando ci sono più variabili $X_1, \ldots, X_p$
- **Variabili di output**:
  - Variabili dipendenti o risposte
  - $Y$
- **Variabili quantitative**: sono variabili con valori numerici, ad esempio la temperatura, l'altezza, il peso, ecc.
- **Variabili qualitative** (o categoriche): sono variabili con valori in una di K classi diverse, ad esempio il genere, la razza, la specie, ecc.
- **Regressione**: predizione di una variabile quantitativa
- **Classificatione**: predizione di una variabile qualitativa

## 7.2 Supervised learning

È l'approccio più utilizzato e consiste nell'addestrare un modello a partire da un insieme di dati etichettati.

**Esempio 7.1.** Prendiamo ad esempio la classificazione di numeri scritti a mano.

- Training set:
  - Immagini conosciute $x$ che rappresentano una certa cifra

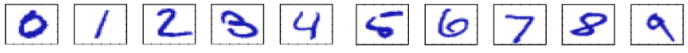


Figura 37: Esempio di immagini di cifre scritte a mano

  - Etichette $t$ che rappresenta la cifra
- Training: Costruisce un modello che mappa immagini a cifre $y(x)$

- Test set: **Nuove** immagini **senza etichette**.
- Testing: Applicazione del modello al test set

Questo approccio può essere rappresentato da una equazione:

$$y = f(x)$$

in cui dato $(x, y)$ si vuole "imparare" $f()$ (function approximation).

## 7.3 Unsupervised learning

Consiste nell'addestrare un modello a partire da un insieme di dati non etichettati.

**Esempio 7.2.** Prendiamo ad esempio il raggruppamento di cellule basato sull'espressione genica.

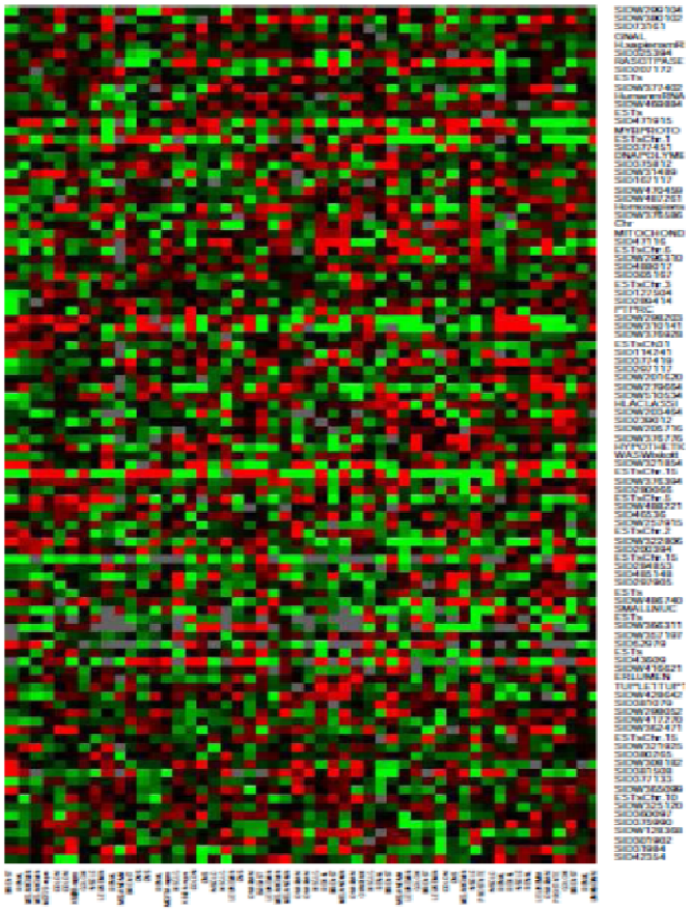


Figura 38: Esempio di clustering di cellule

Le righe rappresentano i geni e le colonne le cellule. Ogni entry $(gene, cell)$ rappresenta il livello di espressione del gene in quella cellula. L'obiettivo è quello di creare un modello che raggruppa per similarità (clustering):

- cellule (colonne)
- geni (righe)
- entrambi

Non si sa il significato del raggruppamento, proprio perchè non si hanno etichette, però si sanno che i dati sono correlati.

Questo approccio può essere rappresentato da una funzione:

$$f(x)$$

Dato $x$ si vuole "imparare" $f()$ come una rappresentazione compatta di $x$ (clustering).

## 7.4 Reinforcement learning

Consiste nell'addestrare un modello a partire da un insieme di dati di interazioni con l'ambiente dando all'agente delle ricompense o penalità.

**Esempio 7.3.** Consideriamo l'esempio di un agente che deve pianificare la traiettoria di un braccio robotico per prendere una pallina.

- Problema di controllo (sequential decision making)
- Segnale di reward per guidare le azioni dell'agente (ad esempio una ricompensa alta quando il braccio raggiunge la pallina)
- Richiede interazione con l'ambiente
- Tecniche statistiche guidate dai dati ma con focus sul controllo e non sull'analisi dei dati

Questo approccio può essere rappresentato da una equazione:

$$y = f(x), z$$

in cui dato $(x, z)$ si vuole "imparare" $f()$ che genera $y$. $x$ è lo stato corrente, $z$ è il segnale di reward.

## 7.5 Predizione

Dato un insieme di input facile da ottenere:

$$X = (X_1, X_2, \ldots, X_p)$$

si vuole predirre un output che è difficile da misurare:

$$Y = f(X) + \varepsilon$$

dove $\varepsilon$ è un **errore casuale**, **indipendente** da $X$ e $f()$ rappresenta un'informazione sistematica di $X$ su $Y$. La **predizione** è la costruzione di un modello $\hat{f}()$ che calcola:

$$\hat{Y} = \hat{f}(X) \text{ dato } X$$

**Esempio 7.4.** Nel seguente esempio la superficie blu è il modello di predizione e si osserva che i valori (punti rossi) non cadono precisamente nel modello a causa dell'errore casuale $\varepsilon$:

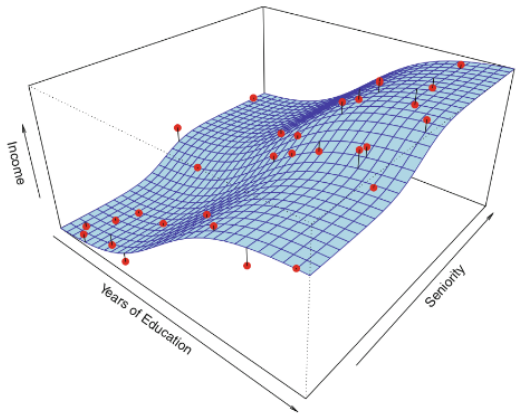



Figura 39: Guadagno come funzione degli anni di educazione e anzianità

### 7.5.1 Accuratezza della predizione

L'accuratezza di una predizione $\hat{Y}$ di $Y$ dipende da due quantità (assumendo che $\hat{f}()$ e $X$ siano fissati):

- **Errore riducibile**: dovuto all'errore nel costruire il modello
- **Errore irriducibile**: dovuto alla variabilità di $\varepsilon$ (errore casuale indipendente da $X$)

$$E(Y - \hat{Y})^2 = E\left[f(X) + \varepsilon - \hat{f}(X)\right]^2 = \underbrace{[f(X) - \hat{f}(X)]^2}_{\text{Errore riducibile}} + \underbrace{Var(\varepsilon)}_{\text{Errore irriducibile}}$$

Dove $E(X)$ con $X$ variabile casuale è il valore atteso della **media** di $X$. Ad esempio la media di una popolazione. $Var(X)$ è la **varianza** di $X$, cioè la misura della dispersione dei valori di $X$ intorno alla media.

### 7.5.2 Inferenza

L'obiettivo è di capire come $Y$ cambia al variare di $X$ (non solo predire $Y$). Bisogna sapere la forma di $f()$. Alcune domande che ci si possono porre sono:

- Quali variabili indipendenti sono associate alla risposta?
- Qual'è la relazione tra la risposta e ogni variabile indipendente?
- Si può approssimare $Y$ con un modello lineare rispetto ad $X$?

## 7.6 Stima di una funzione

Per stimare una funzione $f()$ consideriamo il seguente esempio:

**Esempio 7.5.** Riprendiamo l'esempio 7.4 in cui vogliamo predirre il guadagno annuale di una persona in base agli anni di educazione e all'anzianità:

- Dati di training:
$$n = 30 \text{ osservazioni}$$
- Numero di predittori:
$$p = 2$$
  - Anni di educazione
  - Anzianità
- $x_{ij}$ è il valore per l'osservazione $i$ del predittore $j$
- $y_i$ è il valore della risposta per l'osservazione $i$
- Il training set sono le coppie:
$$(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)$$
dove $x_i = (x_{i1}, x_{i2}, \ldots, x_{ip})^T$ è il vettore dei predittori per l'osservazione $i$.
- L'obiettivo è quello di trovare una stima $\hat{f}()$ tale che:
$$Y \approx \hat{f}(X) \text{ per ogni osservazione } (X, Y)$$

### 7.6.1 Metodi parametrici

I metodi parametrici fanno delle assunzioni sulla forma funzionale di $f()$ facendo assunzioni sulla forma di $f()$. È composta da due passi:

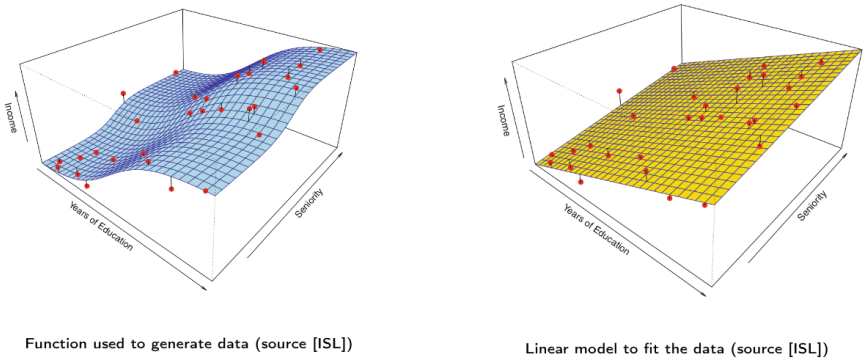
1. Fare un assunzione sulla forma di $f()$, ad esempio assumere che $f()$ sia lineare
   - $f(X) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \ldots + \beta_p X_p$ con $p + 1$ parametri $\beta_0, \ldots, \beta_p$
   - Un modello è completamente identificato dai parametri
2. Trovare un procedimento per **fittare** o **trainare** il modello usando i dati di training
   - Per un modello lineare bisogna stimare i parametri $\beta_0, \ldots, \beta_p$. Bisogna quindi trovare i valori di $\beta_j$ che minimizzano l'errore tali che:
$$Y \approx \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \ldots + \beta_p X_p = \hat{f}(X)$$

Il metodo più comune per trainare modelli lineari è il metodo dei **minimi quadrati** (least square)

**Esempio 7.6.** Riprendendo l'esempio 7.4, possiamo creare un modello lineare:

$$\text{Income} \approx \beta_0 + \beta_1 \cdot \text{ Years of education} + \beta_2 \cdot \text{ Seniority}$$

Si utilizza il metodo dei minimi quadrati per stimare i parametri $\beta_0, \beta_1, \beta_2$ e si ottiene:




Function used to generate data (source [ISL])

Linear model to fit the data (source [ISL])

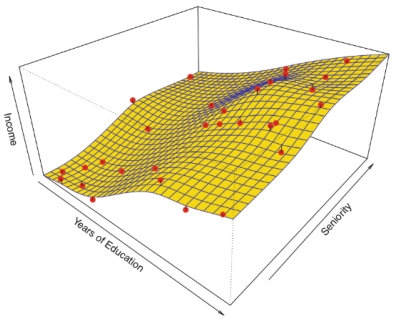
Figura 40: Modello lineare per il guadagno in base agli anni di educazione e all'anzianità

### 7.6.2 Metodi non parametrici

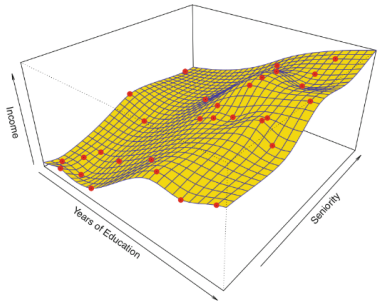
Nei metodi parametrici non si hanno assunzioni su $f()$ e si prova a far fittare il più possibile $f()$ ai dati di training.

- **Vantaggi**:
  - Può approssimare molto bene i dati di training
- **Svantaggi**:
  - Richiede molti dati di training per funzionare bene
  - Può fare overfitting sui dati di training, cioè approssimare troppo bene i dati di training e non generalizzare bene su nuovi dati

**Esempio 7.7.** Riprendendo l'esempio 7.4, possiamo utilizzare un metodo non parametrico per stimare $f()$:



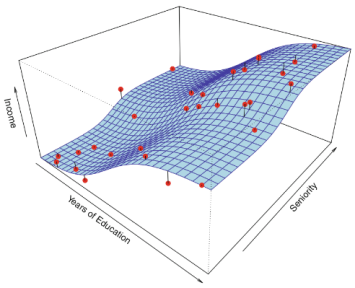

Smooth thin-plate spline fit (source [ISL])



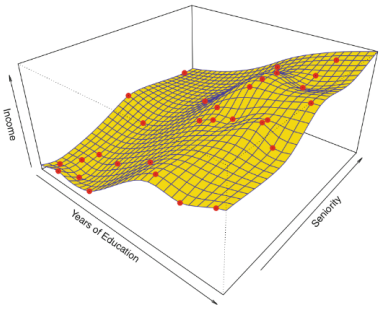

Rough thin-plate spline fit (source [ISL])

Figura 41: Metodo non parametrico per il guadagno in base agli anni di educazione e all'anzianità

Che in confronto con la funzione originale usata per generare i dati di training si nota che è presente overfitting:




Ground truth for income (source [ISL])




Rough thin-plate spline fit (source [ISL])

Figura 42: Confronto tra funzione originale e metodo non parametrico

## 7.7 Accuratezza del modello

### 7.7.1 Interpretabilità e flessibilità

Non esiste un modello migliore in assoluto, ogni modello ha i suoi vantaggi e svantaggi misurati in base a:

- **Interpretabilità**: quanto è facile capire come il modello fa le predizioni
- **Flessibilità**: quanto il modello può adattarsi a diverse forme di dati

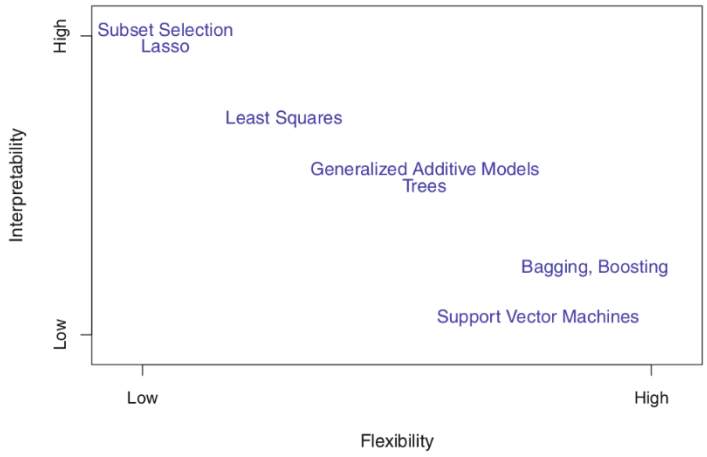



Figura 43: Rappresentazione grafica di flessibilità vs interpretabilità per diversi metodi di statistical learning

### 7.7.2 Capire l'accuratezza

È importante capire che metodo utilizzare per un certo dataset. Alcuni modi per decidere sono:

- **Regressione**: Mean Squared Error (MSE)

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{f}(x_i))^2$$

  dove $y_i$ è il valore reale e $\hat{f}(x_i)$ è la predizione

- **Training MSE**: MSE calcolato sui dati di training
- **Test MSE**: MSE stimato su nuovi dati non visti prima, è il parametro più importante per valutare l'accuratezza del modello

Un metodo che ha training MSE minimo non garantisce che abbia anche test MSE minimo:

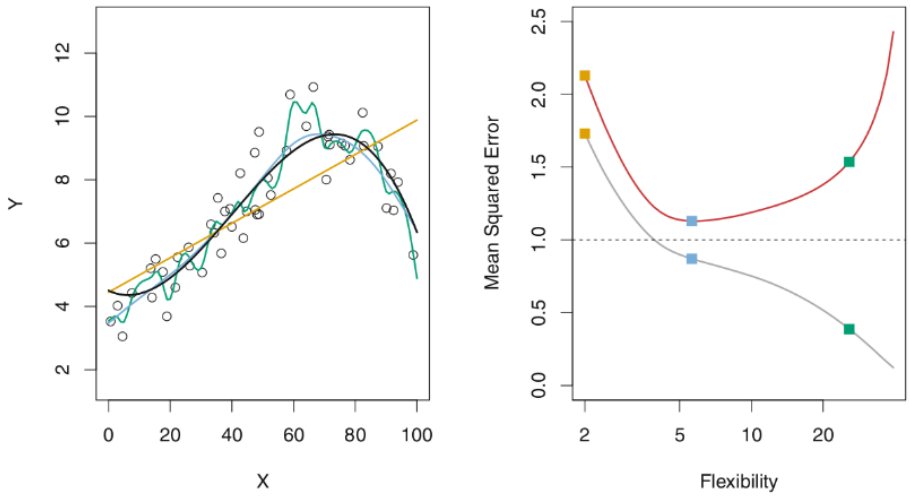



Figura 44: Sinistra: dati simulati dalla curva nera (cerchi); modello lineare (arancione); due smoothing splines (blu e verde). Destra: training MSE (curva grigia), test MSE (curva rossa), minimo possibile test MSE su tutti e tre i metodi (linea tratteggiata grigia). I quadrati rappresentano l'MSE raggiunto dai tre metodi

### 7.7.3 Bias-variance tradeoff

IL valore atteso del **test MSE** per un dato valore $x_0$ può essere sempre scomposto in una somma di 3 componenti:

- Varianza di $\hat{f}(x_0)$: Quantità con cui $\hat{f}(x_0)$ cambierebbe variando il dataset di training. Più flessibile è il metodo, più alta è la varianza
- Squared bias di $\hat{f}(x_0)$: Errore introdotto dalle assunzioni fatte sul modello. Modelli più semplici (ad esempio lineari), hanno un bias più alto
- Varianza dell'errore casuale $\varepsilon$

$$E(y_0 - \hat{f}(x_0))^2 = Var(\hat{f}(x_0)) + [Bias(\hat{f}(x_0))]^2 + Var(\varepsilon)$$

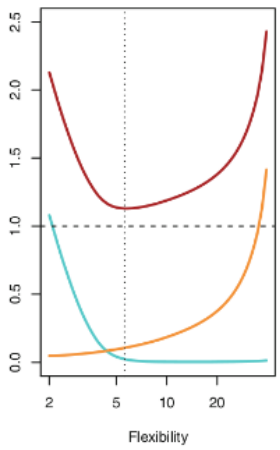



Figura 45: Rappresentazione grafica del bias-variance tradeoff

### 7.7.4 Stimare il test error (cross-validation)

Per stimare il test error si usa l'approccio **Validation set** che consiste nel dividere **casualmente** il dataset in due parti:

- **Training set**: usato per addestrare il modello
- **Validation set** (o hold out set): usato per stimare il test error

Si fitta il modello sul training set e si calcola l'MSE sul validation set. Il validation set **non è** il set test, quindi l'errore è un'approssimazione.




Figura 46: Rappresentazione grafica del validation set

**Svantaggi**:

- La stima del test error può essere molto variabile a seconda di come viene diviso il dataset
- Si usano meno dati per addestrare il modello e quindi si può avere un errore stimato più alto

### 7.7.5 Leave one out cross validation

Per superare gli svantaggi del validation set si può utilizzare il metodo **Leave one out cross validation** che consiste nel lasciare una sola osservazione fuori dal training set e usare quella per il validation set. Si ripete questo processo $n$ volte e si fa la media dell'errore.

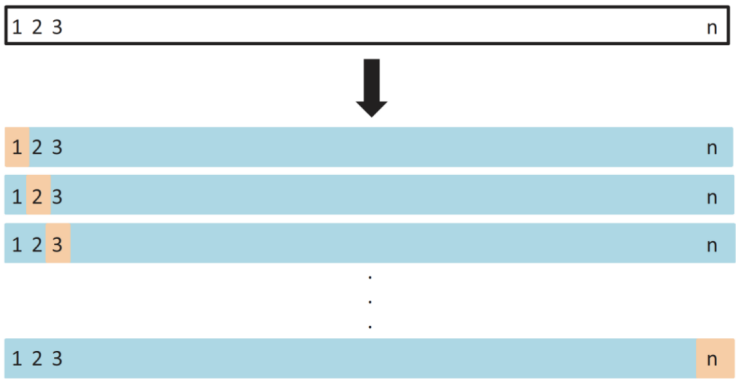



Figura 47: Rappresentazione grafica del leave one out cross validation

**Vantaggi:**

- Si usano tutti i dati per addestrare il modello, quindi si ha meno bias e una stima del test error più accurata
- Non c'è nessuna casualità nella divisione del dataset, quindi se si ripete questo approccio N volte si ottiene sempre lo stesso risultato

Con questo approccio non si trovano i parametri ottimali del modello, ma si ottiene il **tipo di modello** ottimale (ad esempio lineare, polinomiale, ecc...). Successivamente si usano tutti i dati per addestrare il modello con il tipo scelto.

### 7.7.6 K-fold cross validation

L'idea è quella di dividere l'insieme di osservazione in $k$ gruppi (folds). Si usa un fold come validation set e gli altri $k-1$ folds come training set. Si ripete questo processo $k$ volte, cambiando il fold usato come validation set. Si calcola l'errore medio sui $k$ fold:

$$\{MSE_1, MSE_2, \ldots, MSE_3\}$$

$$CV(k) = \frac{1}{k}\sum_{i=1}^{k} MSE_i$$

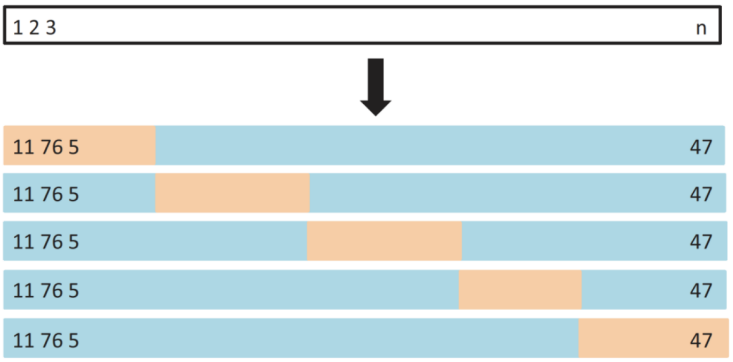



Figura 48: Leave one out cross validation come caso particolare di k-fold cross validation con $k = n$

**Vantaggi:**

- Computazionalmente più efficiente del leave one out cross validation (LOOCV) per grandi dataset, solitamente si usa $k = 5$ o $k = 10$
- Bias-varianza trade-off:
  - Ogni split casuale delle osservazioni addestra un modello su meno osservazioni: bias più alto rispetto a LOOCV
  - k-Fold media l'output di set di osservazioni meno correlate: varianza più bassa rispetto a LOOCV

**Esempio 7.8.** Il test MSE non può essere calcolato in un'applicazione reale, ma si può calcolare se si usano dati simulati. Nell'esempio seguente le curve di Cross Validation hanno la forma generale corretta, ma **sottovalutano** il valore vero del test MSE:

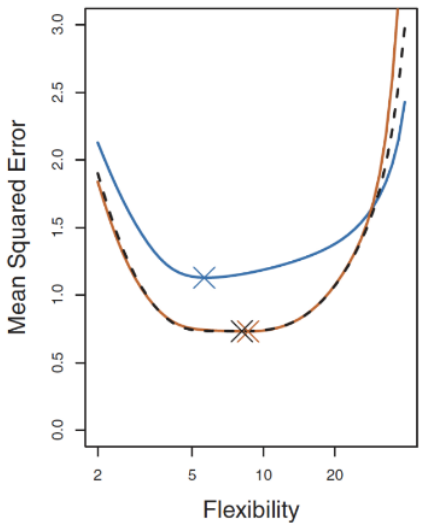



Figura 49: Vero test MSE (blu), LOOCV (nero, tratteggiato), k-fold (arancione, k = 10)

### 7.7.7 Visualizzazione intuitiva del bias-variance tradeoff

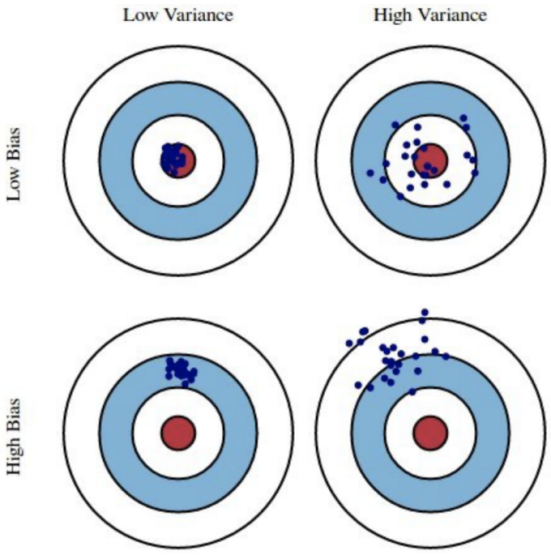



Figura 50: Rappresentazione intuitiva del bias-variance tradeoff

### 7.7.8 Confronto tra cross validation

- **Bias**:
  - **Validation set** può sopravvalutare il test error perchè il training set utilizzato contiene soltanto metà delle osservazioni

  - **LOOCV** fornisce una stima approssimativamente non distorta del test error dato che ogni training set contiene $n-1$ osservazioni (quasi tante quante le osservazioni nell'intero dataset)
  - **k-fold CV** (con $k = 5$ o $k = 10$) porta a un livello intermedio di bias dato che ogni training set contiene $\frac{(k-1)n}{k}$ osservazioni
  - Per ridurre il bias si dovrebbe usare LOOCV.
- **Varianza**:
  - **LOOCV** media gli output di $n$ modelli fittati, addestrati su un insieme di osservazioni quasi identico $\rightarrow$ gli output sono altamente (positivamente) correlati tra loro
  - **k-fold CV** con $k < n$, fa la media gli output di $k$ modelli fittati meno correlati tra loro: la sovrapposizione tra i training set in ogni modello è più piccola
  - **Bias-variance trade-off** è associato alla scelta di $k$ in k-fold cross-validation
  - Solitamente si usa $k = 5$ o $k = 10$ perchè questi valori hanno mostrato empiricamente di fornire stime del test error che non soffrono né di un bias eccessivamente alto né di una varianza molto alta

## 7.8 Regressione lineare

La regressione lineare è un metodo utilizzato per predirre la risposta quantitativa ed è un approccio basilare per il **supervised learning**. I vantaggi sono:

- Sono altamente interpretabili
- Introducono bassa varianza

Gli svantaggi sono:

- Non sono abbastanza potenti per esprimere relazioni complesse tra le variabili
- Bassa accuratezza, alto bias

### 7.8.1 Regressione lineare semplice (univariata)

La funzione lineare univariata è una linea: $y = w_1 x + w_0$ dove:

- $w_0$ è l'intercetta, cioè il valore di $y$ quando $x = 0$
- $w_1$ è il coefficiente angolare

Consideriamo il vettore dei **pesi**:

$$w = \begin{pmatrix} w_0 \\ w_1 \end{pmatrix}$$

e la funzione lineare dei pesi è:

$$h_w(x) = w_1 x + w_0$$

Il problema da risolvere è: dato un insieme di $n$ osservazioni:

$$\{(x_1, y_1), \ldots, (x_n, y_n)\}$$

trovare il valore di $w$ che rappresenta al meglio le osservazioni, cioè si vuole trovare il vettore $w$ che minimizza la **perdita**. Solitamente la perdita è definita come la somma dei minimi quadrati degli errori ($L_2$ norm) su tutte le osservazioni (Residual Sum of Squares oppure Least Squares)

$$\text{Loss}(h_w) = \sum_{i=1}^{n} L_2(y_i, h_w(x_i)) = \sum_{i=1}^{n} (y_i - h_w(x_i))^2 = \sum_{i=1}^{n} (y_i - (w_1 x_i + w_0))^2$$

**Esempio 7.9.** Consideriamo il seguente dataset che rappresenta le vendite di un prodotto in 200 mercati diversi, insieme al budget pubblicitario per il prodotto in ciascuno di questi mercati per tre diversi media: TV, radio e giornale. Qui consideriamo solo la pubblicità in TV.

$$sales = w_1 \cdot \text{TV} + w_0$$

dove:

- $w_0 = 7.03$
- $w_1 = 0.0475$

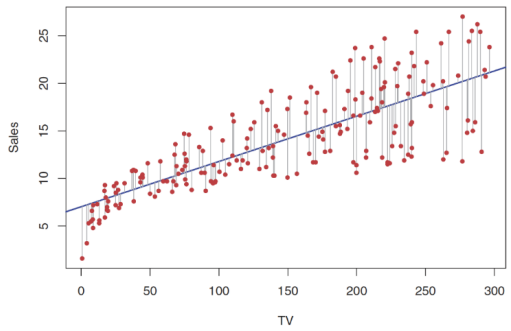



Figura 51: Esempio di regressione lineare semplice

### 7.8.2 Trovare i pesi ottimali

Per trovare i pesi ottimali $w*$ si usa la seguente formula:

$$w* = \text{argmin} \text{Loss}(h_w) = \text{argmin} \sum_{i=1}^{n} (y_i - (w_1 x_i + w_0))^2$$

Questa funzione è minimizzata quando le derivate parziali rispetto a $w_0$ e $w_1$ sono uguali a zero:

$$\frac{\partial \text{Loss}(h_w)}{\partial w_0} = \frac{\partial}{\partial w_0} \sum_{i=1}^{n} (y_i - (w_1 x_i + w_0))^2 = 0$$
$$\frac{\partial \text{Loss}(h_w)}{\partial w_1} = \frac{\partial}{\partial w_1} \sum_{i=1}^{n} (y_i - (w_1 x_i + w_0))^2 = 0$$

Queste equazioni hanno una sola soluzione che può essere calcolata come:

$$\hat{w}_1 = \frac{n \left(\sum_{i=1}^{n} x_i y_i\right) - \left(\sum_{i=1}^{n} x_i\right) \left(\sum_{i=1}^{n} y_i\right)}{n \left(\sum_{i=1}^{n} x_i^2\right) - \left(\sum_{i=1}^{n} x_i\right)^2}$$
$$\hat{w}_0 = \frac{\left(\sum_{i=1}^{n} y_i\right) - \hat{w}_1 \left(\sum_{i=1}^{n} x_i\right)}{n}$$

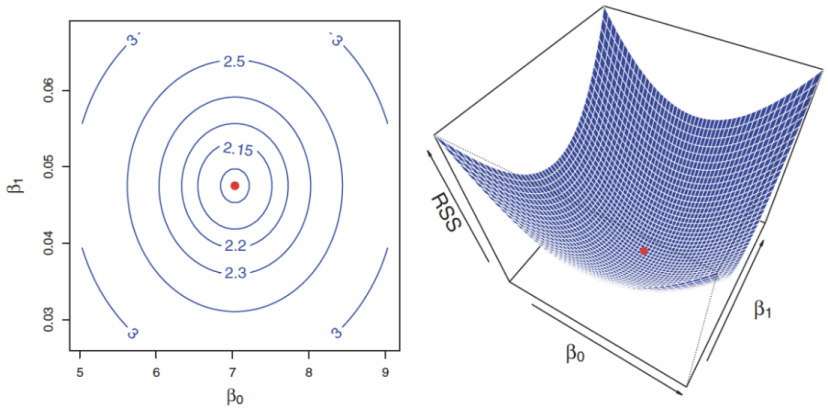



Figura 52: Spazio dei pesi per la regressione lineare semplice con $w = (\beta_0, \beta_1)$

Questo metodo **garantisce** che la funzione di loss sia convessa, quindi si ha sempre un singolo punto di minimo e di conseguenza si trova sempre la soluzione ottimale. Questo non è sempre garantito per altri modelli e si devono usare altri approcci come il gradient descent.

### 7.8.3 Gradient descent

Il gradient descent è un algoritmo iterativo per trovare un **minimo locale**. L'algoritmo è il seguente:

$$\begin{cases} \text{inizializza vettore dei pesi } w \text{ con un punto dello spazio dei pesi} \\ \text{Ripeti fino a convergenza:} \\ \quad w_i \leftarrow w_i - \alpha \frac{\partial}{\partial w_i} \text{Loss}(h_w) \ \forall w_i \in w \end{cases}$$

dove $\alpha$ è il **learning rate** che controlla la dimensione del passo ad ogni iterazione.

**Esempio 7.10.** Per la regressione lineare semplice con loss $L_2$ abbiamo:

$$w_0 \leftarrow w_0 + \alpha \sum_{i=1}^{n} (y_i - h_w(x_i))$$

$$w_1 \leftarrow w_1 + \alpha \sum_{i=1}^{n} (y_i - h_w(x_i)) \cdot x_i$$

### 7.8.4 Stochastic gradient descent

Se si usano le regole di aggiornamento del gradient descent per ogni osservazione si ha solamente una sola **learning epoch**, cioè un solo passaggio attraverso tutte le osservazioni. Il gradient descent stocastico consiste nel fare l'aggiornamento dei pesi selezionando un piccolo sottoinsieme di osservazioni (minibatch) ad ogni iterazione. Questo permette di calcolare l'aggiornamento in parallelo per ogni minibatch rendendo l'altoritmo più veloce. Il problema di questo approccio è che la **convergenza non è garantita**. La convergenza si potrebbe raggiungere con una procedura che riduce $\alpha$. Questo approccio è usato principalmente per funzioni loss non convesse e nonostante non garantisca l'ottimalità ha buone performance.

## 7.9 Reti neurali

L'idea principale di una Artificial Neural Network (ANN) è di modellare una funzione **non lineare** partendo da una combinazione lineare di input come feature derivate (funzione applicata agli input).

### 7.9.1 Feed-forward neural network

Una rete neurale feed-forward è composta da un modello di regressione (o classificazione) a due stadi rappresentato da un grafo aciclico diretto (DAG) in cui ogni nodo rappresenta l'unità di computazione (neurone). Il modello è composto da:

- $X$ **Inputs**: $\{X_1, \ldots, X_p\}$
- $Z$ **Hidden layer(s)**: $\{Z_1, \ldots, Z_m\}$
- $Y$ **Output**: $\{Y_1, \ldots, Y_k\}$

Gli archi rappresentano le connessioni tra i nodi e ad ogni arco è associato un peso $w_{ij}$ che rappresenta l'importanza della connessione tra il nodo $i$ e il nodo $j$.

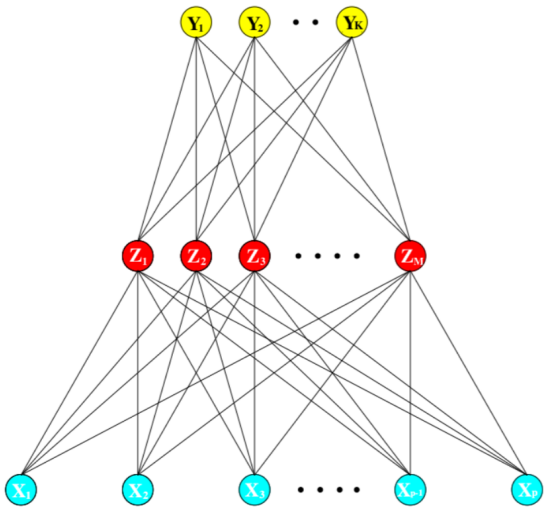



Figura 53: Rappresentazione grafica di una rete neurale feed-forward

Un certo neurone (nodo) si **attiva** quando l'input **supera un certo limite**. Ogni neurone calcola una **funzione di attivazione** (tipicamente non lineare) $h$:

$$h(w^T x + b)$$

dove:

- $w$ è il vettore dei pesi associati agli archi in ingresso
- $x$ è il vettore degli input
- $b$ è il bias (intercetta)

Ogni layer ha la propria funzione di attivazione $h$, solitamente:

- **Hidden units**: $\sigma$ che tipicamente è:

$$\sigma = \frac{1}{1 + e^{-x}} \text{ (sigmoide)}$$

- **Output units**: $g$

### 7.9.2 Caolcolo di una rete neurale

Il calcolo di una rete neurale si fa sommando tutti i pesi in ingresso moltiplicati per gli output dei nodi di partenza. Questo valore viene passato alla funzione di attivazione per calcolare l'output del nodo di arrivo.

- **Hidden units**:

$$z_j = h_1\left(w_j^{(1)} x + b_j^{(1)}\right)$$

- **Output units**:

$$y_k = h_2\left(w_k^{(2)} z + b_k^{(2)}\right)$$

- **In generale** gli output si calcolano come:

$$y_k = h_2 \left( \sum_j w_{kj}^{(2)} h_1 \left( \sum_i w_{ji}^{(1)} x_i + b_j^{(1)} \right) + b_k^{(2)} \right)$$

dove:

- $h_1, h_2$ sono le funzioni di attivazione
- $w^{(1)}, w^{(2)}$ sono i pesi associati agli archi
- $b^{(1)}, b^{(2)}$ sono i bias, solitamente sono nodi costanti a 1

Un esempio di rete neurale feed-forward con 2 input, 2 hidden units e 1 output è:

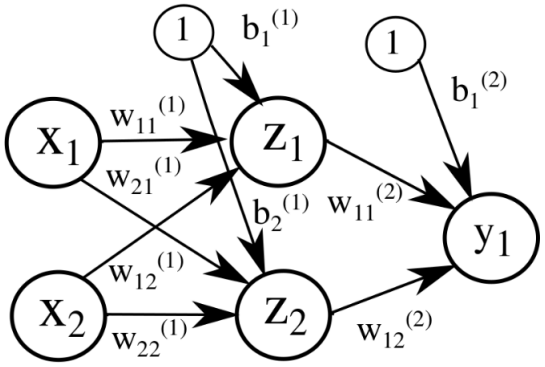



Figura 54: Esempio di rete neurale feed-forward

In una rete neurale ciò che si vuole stimare sono i pesi $w$ associati agli archi.

**Esempio 7.11.** Un esempio concreto di rete neurale feed-forward è il seguente:

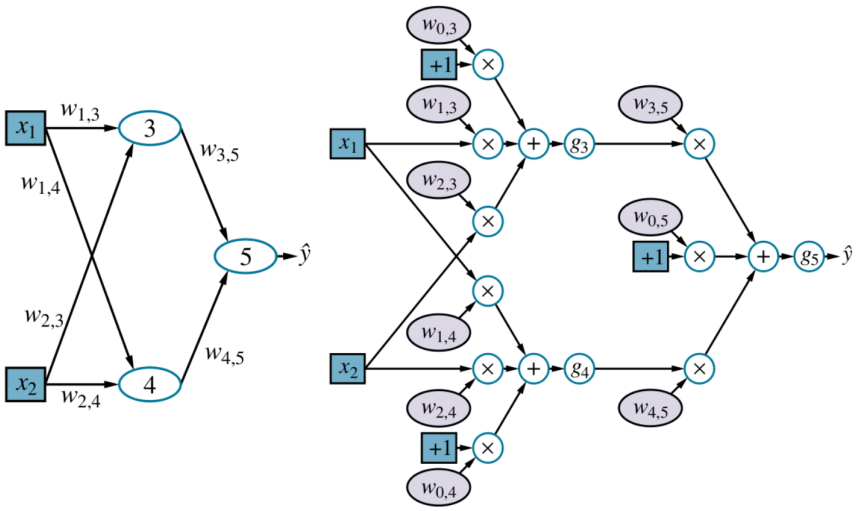



Figura 55: Esempio di rete neurale feed-forward con funzioni di attivazione sigmoide

### 7.9.3 Funzione di attivazione

Ci sono diversi tipi di funzioni di attivazione:

- **Threshold**: Questa funzione non è derivabile in $x = 0$, quindi non è usata nelle reti neurali moderne siccome non è possibile calcolare il gradient descent e quindi non è possibile stimare i pesi.

$$h(x) = \begin{cases} 1 & \text{se } x \geqslant 0 \\ -1 & \text{altrimenti} \end{cases}$$

- **Sigmoid**:

$$h(x) = \sigma(x) = \frac{1}{1 + e^{-x}}$$

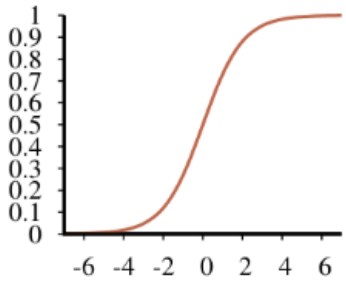



Figura 56: Funzione sigmoide

La derivata prima di questa funzione è facile da calcolare e vale:

$$\frac{d\sigma(x)}{dx} = \sigma(x)(1 - \sigma(x))$$

È sempre compresa tra 0 e 1.

- **Tanh** (tangente iperbolica):

$$h(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

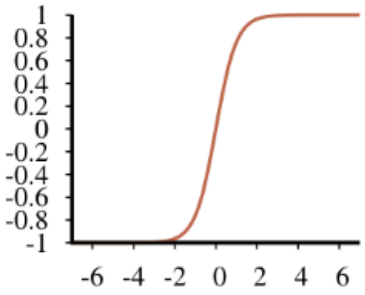



Figura 57: Funzione tanh

La derivata prima di questa funzione vale:

$$\frac{d\tanh(x)}{dx} = 1 - \tanh^2(x)$$

È sempre compresa tra 0 e 1.

- **Gaussian**:

$$h(x) = e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

- **Identity**:

$$h(x) = x$$

- **Rectified linear (ReLU)**: È la funzione di attivazione più usata nelle reti neurali moderne.

$$h(x) = max\{0, x\}$$

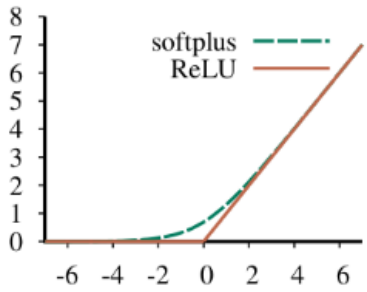



Figura 58: Funzione ReLU

Questa funzione non è derivabile in $x = 0$ e il suo valore è 1 per $x > 0$.

La funzione **softplus** è una versione smussata della ReLU (la approssima):

$$h(x) = \log(1 + e^x)$$

per $x < 0$ si ha $h(x) \approx 0$ e per $x > 0$ si ha $h(x) \approx x$. La sua derivata prima è uguale alla funzione sigmoide ed è sempre compresa tra 0 e 1:

$$\frac{dh(x)}{dx} = \frac{1}{1 + e^{-x}} = \sigma(x)$$

### 7.9.4 Proprietà universale di approssimazione

**Teorema 7.1** (Hornik et al., 1989, Cybenko, 1989)**.** Una rete neurale feed-forward con un layer di **output lineare** e **almeno un** hidden layer con una funzione di attivazione "squashing" (sigmoide/tanh/gaussian) **può approssimare qualsiasi funzione continua (definita su un insieme chiuso e limitato di $\mathbb{R}^n$) arbitrariamente bene**, a patto che la rete abbia un numero sufficiente di hidden units.

### 7.9.5 Addestramento di una rete neurale

L'addestramento di una rete neurale consiste nel trovare i pesi $w$ basandosi su un insieme di dati di training. L'insieme di pesi è composto da tutti i pesi associati agli archi di ogni layer:

$\{\alpha_{m0}, \alpha_m; m = 1, \ldots, M\}$ $M \cdot (p + 1)$ pesi tra input e hidden layer
$\{\beta_{k0}, \beta_k; k = 1, \ldots, K\}$ $K \cdot (M + 1)$ pesi tra hidden layer e output

Per la **regressione**, cioè quando la risposta è quantitativa, si usa la **somma dei minimi quadrati** come misura di fit:

$$RSS = \sum_{k=1}^{K} \sum_{i=1}^{N} (y_{ik} - f_k(x_i))^2$$

### 7.9.6 Back-propagation

Siccome la rete neurale ha una forma composizionale, cioè la funzione di output è una composizione di funzioni derivate dagli hidden layer, si può sfruttare questa struttura per calcolare efficientemente le derivate della funzione di loss rispetto ai pesi usando l'algoritmo di **Back-propagation**:

- Usare i pesi correnti per calcolare $f_k(x_i)$, calcolo in avanti (forward computation) sulla rete
- Eseguire una fase all'indietro (backward phase) calcolando gli errori nell'ultimo layer e propagando l'errore al layer precedente
- Aggiornare i pesi usando il gradient descent

$$E(W) = \sum_{i=1}^{N} E_i = \sum_{i=1}^{N} \sum_{k=1}^{K} (y_{ik} - f_k(x_i))^2$$

Quindi considerando un hidden unit $z_m i$:

$$z_m i = \sigma\left(\alpha_{m0} + \alpha_m^T x_i\right) \quad z_i = \{z_{1i}, \ldots, z_{Mi}\}$$

Si può calcolare la derivata parziale dell'errore rispetto ai pesi come:

$$\frac{\partial E_i}{\partial \beta_{km}} = -2\left(y_{ik} - f_k(x_i)\right) \cdot g_k'\left(\beta_k^T z_i\right) \cdot z_{mi}$$

$$\frac{\partial E_i}{\partial \alpha_{ml}} = \sum_{k=1}^{K} -2\left(y_{ik} - f_k(x_i)\right) \cdot g_k'\left(\beta_k^T z_i\right) \cdot \beta_{km} \cdot \sigma'\left(\alpha_m^T x_i\right) \cdot x_{il}$$

La parte in blu rappresenta l'errore propagato all'indietro (back propagation). Successivamente si aggiornano i pesi usando il gradient descent all'iterazione $(r + 1)$:

$$\beta_{km}^{(r+1)} = \beta_{km}^{(r)} - \gamma_r \sum_{i=1}^{N} \frac{\partial E_i}{\partial \beta_{km}^{(r)}}$$

$$\alpha_{ml}^{(r+1)} = \alpha_{ml}^{(r)} - \gamma_r \sum_{i=1}^{N} \frac{\partial E_i}{\partial \alpha_{ml}^{(r)}}$$

Il back propagation è molto semplice e **locale**, cioè ogni unità calcola la propria parte di errore e la invia alle unità collegate a essa, quindi è facilmente parallelizzabile. Il training può essere effettuato in due modi:

- **Batch**: Un solo aggiornamento dei pesi sommando i gradienti dell'errore su tutte le osservazioni.
- **Online**: Aggiornamento dei pesi per ogni osservazione.

$\gamma_r$ è la costante di **batch learning** che dovrebbe diminuire a 0 per $r \to \infty$. Il valore che garantisce la convergenza è:

$$\gamma_r = \frac{1}{r}$$

### 7.9.7 Deep neural networks

Le reti neurali profonde (DNN) sono reti neurali con più di un hidden layer. Il vantaggio è quello di avere espressività maggiore (più compatta rispetto ad un layer con tanti nodi), ma i problemi da affrontare sono:

- Addestrare la rete neurale (back-propagation diventa più complesso)
- Evitare l'overfitting (regolarizzazione più complessa)

Il problema principale delle deep neural networks (con funzioni "squashing") è il **vanishing gradient**, cioè durante il back-propagation i gradienti diventano sempre più piccoli man mano che si procede verso gli strati più bassi della rete. Questo avviene perchè le funzioni "squashing" hanno derivate comprese tra 0 e 1, quindi moltiplicando più derivate si ottiene un valore sempre più piccolo. Questo è il motivo per cui si usano funzioni di attivazione come la ReLU che non soffrono di questo problema.

**Esempio 7.12.** Prendiamo in considerazione la funzione $\tanh$ e osserviamo la derivata:

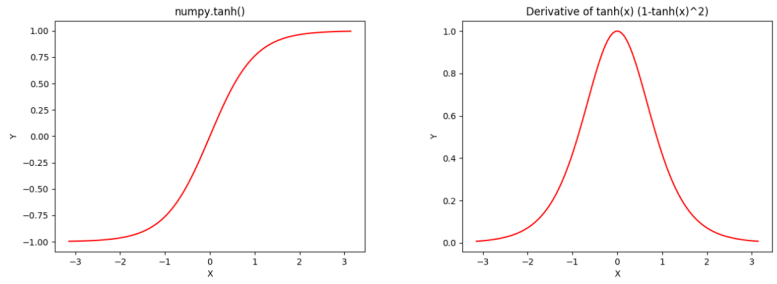



Figura 59: Derivata della funzione $\tanh$

Questo avviene anche per la funzione **softplus** che anch'essa ha derivata minore di 1 nonostante sia un'approssimazione della ReLU.

**Esempio 7.13.** Consideriamo un dataset di immagini con label che indicano la linea d'acqua, l'addestramento della rete neurale consiste nel trovare i pesi che riescono a dare in output un pixel nero dove c'è acqua e un pixel bianco dove non c'è acqua:

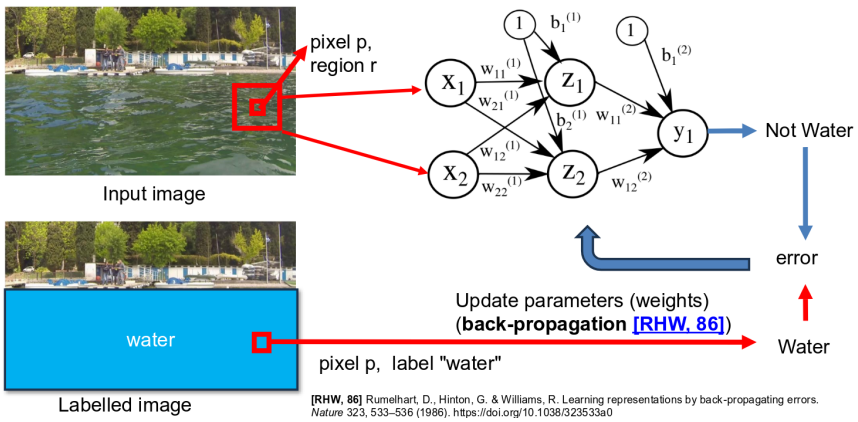



Figura 60: Esempio di addestramento di una rete neurale per il riconoscimento della linea d'acqua

## 7.10 Generative AI

Generative AI è un campo dell'intelligenza artificiale che si concentra sulla creazione di modelli in grado di generare nuovi dati simili a quelli su cui sono stati addestrati. L'idea principale è quella di rappresentare i concetti come vettori (**embedding**) in uno spazio multidimensionale, in modo che le relazioni tra i concetti possano essere

catturate attraverso operazioni aritmetiche sui vettori. I concetti simili sono vicini nello spazio degli embedding.

**Esempio 7.14.** Un esempio di word embedding, con alcune operazioni sui vettori è il seguente:

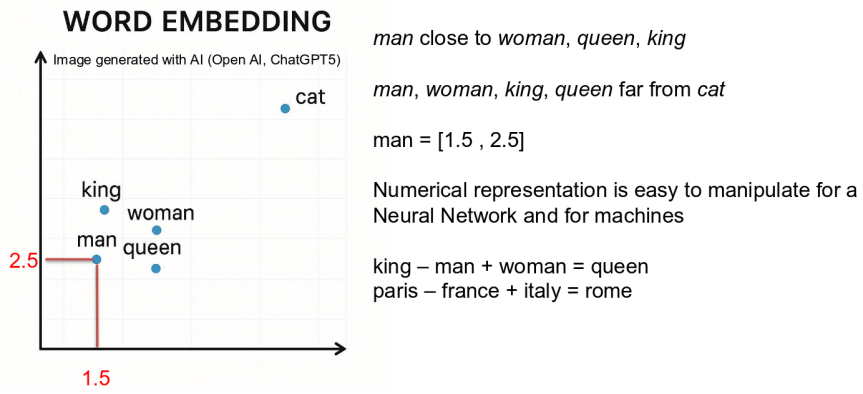



Figura 61: Esempio di word embedding

### 7.10.1 Rappresentare i concetti tramite embedding

Per trovare la corretta rappresentazione nello spazio degli embedding dei concetti si usa la **contrastive learning**, che consiste nel dare esempi positivi e negativi al modello. Gli esempi positivi sono coppie di concetti simili, mentre gli esempi negativi sono coppie di concetti non simili. I concetti possono essere di diversi tipi, come immagini, testo, audio, video, ecc...

**Esempio 7.15.** Un esempio di contrastive learning è il seguente:

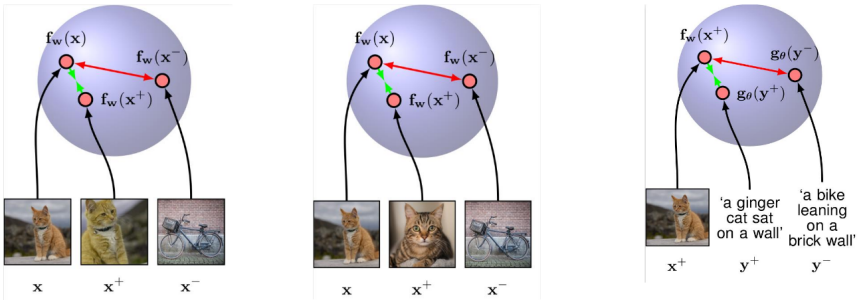



Figura 62: Esempio di contrastive learning per immagini

### 7.10.2 Attenzione

L'importanza di un conncetto dipende dal contesto in cui viene usato. Ad esempio:

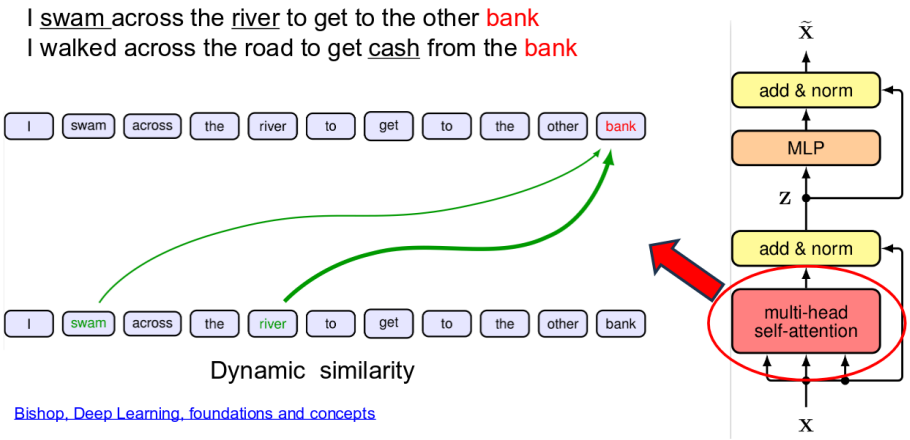



Figura 63: Esempio di attenzione nei modelli di linguaggio

Questo meccanismo è chiamato **attenzione**.

I **transformer** sono un'architettura di rete neurale basata sul meccanismo di attenzione. L'idea principale è quella di permettere al modello di "prestare attenzione" a diverse parti dell'input in base al contesto.

### 7.10.3 Language models

I **Language Models** (LM) sono modelli che trasformano i concetti in embedding di testo in modo da predirre la parola successiva in una frase. Il modello si basa su:

- Embedding e tokenizzazione per rappresentare l'input
- Transformer, attention per rappresentare il contesto
- **Self-supervised learning** per addestrare il modello su grandi quantità di testo senza bisogno di label

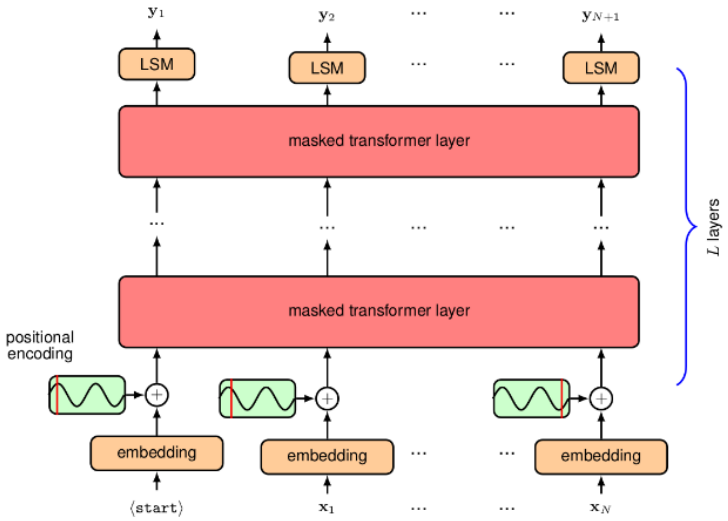



Figura 64: Rappresentazione grafica di un language model

Un Large Language Model (LLM) è un LM con un numero molto grande di parametri, almeno 10 miliardi di parametri.

## 7.11 Reinforcement learning

Il **Reinforcement Learning** (RL) è un paradigma di apprendimento automatico in cui un agente apprende a prendere decisioni ottimali, massimizzando una ricompensa, interagendo con un ambiente.

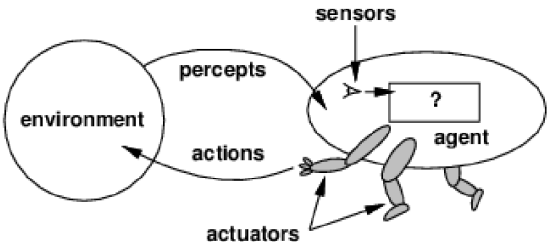



Figura 65: Rappresentazione grafica del reinforcement learning

I concetti chiave del RL sono:

- **Trial and error**: l'agente esplora l'ambiente e apprende dalle conseguenze delle sue azioni
- **Delayed reward**: le azioni possono avere conseguenze a lungo termine

L'obiettivo del RL è stimare a lungo termine la **funzione di valore** $V(s)$, e trovare la **policy** ottimale $\pi(s)$ che massimizza la ricompensa attesa.

Il reinforcement learning è un MDP (Markov Decision Process), ma:

- Non si conoscono gli stati positivi o negativi (non si conosce $R(s, a, s')$)
- Non si conoscono le transizioni tra stati (non si conosce $T(s, a, s')$)

Si deve quindi provare ogni azione e collezionare la ricompensa.

### 7.11.1 Conoscenza del modello

Ci sono due approcci principali per fare reinforcement learning:

- **Model-based**: sono metodi che cercano di **stimare il modello dell'ambiente**. Questi metodi evitano stati e azioni che portano a ricompense negative e quindi usano meno step per raggiungere l'obiettivo con un utilizzo più efficiente dei dati.
- **Model-free**: sono metodi cercano di **imparare la Q-function e la policy ottimale** direttamente dalle esperienze dell'agente. Questi metodi sono più semplici da implementare perchè non richiedono un modello, ma hanno bisogno di più dati per apprendere una buona policy. Una diretta conseguenza è che questi metodi hanno meno bias rispetto ai metodi model-based.

**Esempio 7.16.** Si vuole calcolare l'età attesa per una classe data la distribuzione di probabilità:

$$\mathbb{E}[A] = \sum_a P(a) \cdot a$$

- **Model-based**: stimare $\hat{P}(a)$:

  $$\hat{P}(a) = \frac{\text{numero di studenti con età } a}{\text{numero totale di studenti}}$$

  e quindi:

  $$\mathbb{E}[A] \approx \sum_a \hat{P}(a) \cdot a$$

  Funziona perchè si impara il modello corretto.

  Oltre a stimare il modello il duale è la stima della funzione di transizione $T(s, a, s')$ che si può calcolare dati i samples con le relative azioni (traiettoria):

  $$s_0, a_0, s_1, a_1, s_2, a_2, \ldots$$

  Si stima $\hat{T}(s, a, s')$:

  $$\hat{T}(s, a, s') = \frac{\text{count}(s_{t+1} = s', a_t = a, s_t = s)}{\text{count}(s_t = s, a_t = a)}$$

  Lo stesso vale per la stima della funzione di ricompensa

- **Model-free**: campionare N studenti e calcolare la media delle età:

$$\mathbb{E}[A] \approx \frac{1}{N} \sum_{i=1}^{N} a_i$$

Funziona perchè la media campionaria converge alla media vera al crescere di N.

### 7.11.2 Metodi model-based

Un algoritmo per l'approccio model-based per il reinforcement learning è il seguente:

```
Require: A, S, S_0
Ensure: T_hat, R_hat, pi_hat
  Initialize T_hat, R_hat, pi_hat
  repeat
    Execute pi_hat for a learning episode
    Acquire a sequence of tuples <(s, a, s', r )>
    Update T_hat and R_hat according to tuples <(s, a, s', r )>
    Given current dynamics compute a policy (e.g., VI or PI)
  until termination condition is met
```

### 7.11.3 Metodi model-free

L'obiettivo dei metodi model-free è quello di stimare un'expectation pesata per il modello senza il modello, quindi partendo dai campioni:

$$x_i \sim P(x),\ \mathbb{E}[f(x)] \approx \frac{1}{N} \sum_{i} f(x_i)$$

Dove $x_i \sim P(x)$ indica che i campioni sono estratti dalla distribuzione di probabilità $P(x)$. Alla fine si vuole stimare una funzione di valore data la policy $\pi$:

- Si esegue $\pi$ per alcuni episodi di learning
- Si calcola la somma dei reward scontati **ogni volta che si visita uno stato**
- Si calcola la media dei reward scontati per ogni campione

### 7.11.4 Sample-based policy evalueation

L'obiettivo è quello di **migliorare** la funzione di valore $V$ considerando la Bellman Update, data una policy $\pi$:

$$V_\pi^{k+1}(s) = \sum_{s'} T(s, \pi(s), s') \left( R(s, \pi(s), s') + \gamma V_\pi^k(s') \right)$$

### 7.11.5 Temporal Difference Learning

Il Temporal Difference Learning permette di usare la Sample-based policy evaluation per **imparare da ogni esperienza** senza aspettare la fine dell'episodio. Questo viene fatto calcolando una **running average**:

- Si parte dal campione:

$$sample = R(s, \pi(s), s') + \gamma V_\pi(s')$$

- Si aggiorna il valore come:

$$V_\pi(s) = (1 - \alpha)V_\pi(s) + \alpha(sample - V_\pi(s))$$

- $\alpha$ deve decrescere nel tempo per garantire la convergenza, l'opzione più semplice è mettere $\alpha = \frac{1}{n}$ dove $n$ è il numero di volte che lo stato $s$ è stato visitato.

Quindi:

$$V_\pi(s) \leftarrow (1 - \alpha)V_\pi(s) + \alpha\left(R(s, \pi(s), s') + \gamma V_\pi(s')\right)$$

L'obiettivo è calcolare una policy, ma questo non si può fare perchè per calcolarla bisognerebbe trovare l'azione che massimizza $Q(s, a)$:

$$\pi(x) = argmax_a Q(s, a)$$

dove:

$$Q(s, a) = \sum_{s'} T(s, a, s')\left(R(s, a, s') + \gamma V(s')\right)$$

Non si può usare direttamente $V$ per calcolare $\pi$. L'idea è quella di stimare direttamente i valori di $Q(s, a)$.

### 7.11.6 Q-learning

Il Q-learning è un algoritmo di reinforcement learning che permette di stimare direttamente la Q-function $Q(s, a)$ senza conoscere il modello dell'ambiente.

$$Q_{k+1}(s, a) = \sum_{s'} T(s, a, s')\left(R(s, a, s') + \gamma \max_{a'} Q_k(s', a')\right)$$

Questa equazione trova iterativamente il valore ottimo di Q.

### 7.11.7 Sample based Q-learning

Dati dei campioni $(s, a, s', r)$ si può aggiornare la Q-function in base alla vecchia stima $Q(s, a)$ considerando il nuovo campione:

$$sample = R(s, a, s') + \gamma \max_{a'} Q(s', a')$$

Si incorpora la nuova stima in una running average:

$$Q(s, a) \leftarrow (1 - \alpha)Q(s, a) + \alpha(R(s, a, s') + \gamma \max_{a'} Q(s', a'))$$

Il valore di $\alpha$ rappresenta il **learning rate**, cioè il peso che viene dato al nuovo campione rispetto alla vecchia stima. Il valore di $\alpha$ deve decrescere nel tempo per garantire la convergenza. Questo non è un modello, è un esperienza, cioè si esegue un azione e si osserva la conseguenza.

**Proprietà**:

- Il Q-learning converge alla policy ottima se:
  - Si esplora abbastanza
  - se $\alpha$ diventa abbastanza piccolo, ma non troppo velocemente. Tipicamente il valore usato è $\alpha = \frac{1}{n(s,a)}$, dove $n(s, a)$ è il numero di visite alla coppia $(s, a)$
- La selezione delle azioni non impatta la convergenza. Esiste una strategia chiamata **Off-policy learning** che permette di imparare la policy ottimale senza seguirla, ma per garantire la convergenza bisogna visitare ogni coppia stato-azione infinite volte.

Lo pseudocodice del Q-learning è il seguente:

```
Initialize Q(s, a), for all S, a in A(s), arbitrarily, and Q(
    terminal-state, *) = 0
Repeat (for each episode):
  Initialize s
  Repeat (for each step of episode):
    Choose a from s using policy derived from Q (e.g., epsilon-
    greedy)
    Take action a, observe r, s'
    Q(s, a) <- Q(s, a) + alpha [r + gamma max_a' Q(s', a') - Q(s, a
    )]
    s <- s'
  until s is terminal
```

### 7.11.8 SARSA: alternativa on-policy al Q-learning

Questo algoritmo è simile al Q-learning, ma segue la policy corrente per scegliere l'azione successiva $a'$ invece di scegliere l'azione che massimizza $Q(s', a')$. Il Q-value viene fatto sapendo quale azione si prenderà nel prossimo stato.

```
Initialize Q(s, a), for all S, a in A(s), arbitrarily, and Q(
    terminal-state, *) = 0
  Repeat (for each episode):
    Initialize s
    Choose a from s using policy derived from Q (e.g., epsilon-
    greedy)
    Repeat (for each step of episode):
      Take action a, observe r, s'
      Choose a' from s' using policy derived from Q (e.g., epsilon-
    greedy)
      Q(s, a) <- Q(s, a) + alpha [r + gamma Q(s', a') - Q(s, a)]
      s <- s'; a <- a'
    until s is terminal
```

Questo algoritmo converge solo se la policy usata per scegliere le azioni converge alla policy greedy e ogni coppia stato-azione viene visitata infinite volte.

## 7.12 Esplorazione vs Sfruttamento

Un problema fondamentale nel reinforcement learning è il trade-off tra **esplorazione** e **sfruttamento**:

- **Esplorazione**: consiste nel provare nuove azioni per scoprire nuove strategie che potrebbero portare a ricompense maggiori in futuro.

- **Sfruttamento**: consiste nel scegliere le azioni che si conoscono per massimizzare la ricompensa immediata.

La scelta dipende dal contesto e dall'obiettivo dell'agente.
Ci sono due approcci principali:

- $\varepsilon$-**greedy**:
  La politica $\varepsilon$-greedy sceglie l'azione migliore di sempre, ma ogni tanto (con probabilità $\varepsilon$) sceglie un'azione a caso. Questo metodo è off-policy.

- **Softmax** (o Boltzmann):
  Si sceglie un'azione in base ad una distribuzione di probabilità:

$$p(a) = \frac{e^{Q(s,a)/T}}{\sum_{a'} e^{Q(s,a')/T}}$$

  T è il parametro di temperatura che controlla il livello di esplorazione:

  - Quando la "temperatura" T è alta, tutte le azioni hanno probabilità simili di essere scelte (si esplora di più)
  - Quando la temperatura T è bassa, l'azione con il valore $Q(s, a)$ più alto ha una probabilità maggiore di essere scelta (si sfrutta di più)

### 7.12.1 Funzioni di esplorazione

L'idea è quella di aggiungere un bonus di esplorazione alla ricompensa per incentivare l'agente a esplorare nuove azioni. Si esplorano le aree in cui non si è sicuri che siano negative (optimism in face of uncertainty). La funzione di esplorazione è definita considerando:

- Una stima $u$
- Un numero di visite $n$
- Il calcolo di una funzione $f(u, n) = u + \frac{k}{n}$
- Update regolare della Q-function:

$$Q(s, a) = (1 - \alpha)Q(s, a) + \alpha(R(s, a, s') + \gamma \max_{a'} Q(s', a'))$$

- Update modificato con bonus di esplorazione:

$$Q(s, a) = (1 - \alpha)Q(s, a) + \alpha(R(s, a, s') + \gamma \max_{a'} f(Q(s', a'), N(s, a)))$$

  dove:

  - $N(s', a')$ è il numero $n$ di volte che una coppia stato-azione $(s', a')$ è stata visitata
  - $f(Q(s', a'), N(s, a))$ è la funzione di esplorazione che aggiunge un bonus alla stima di $Q(s', a')$ in base a $N(s, a)$
  - $k$ è un parametro fissato

## 7.13 Deep reinforcement learning

Nel mondo reale gli stati e le azioni sono spesso continui e in grandi quantità e quindi non si riesce a rappresentare le funzioni chiave per il reinforcement learning: $(\pi(s), V(s), Q(s,a))$ come tabelle. Per risolvere questo problema si possono approssimare usando:

- Approssimazione lineare
- Approssimazione tramite rete neurale (Deep Reinforcement Learning)
- Approssimazione con Deep Q-Networks (DQN) in cui si approssima $Q(s,a)$ usando una Deep Neural Network

### 7.13.1 Gradient Q-Learning

Per approssimare $Q(s,a)$ con una funzione parametrica $Q_w(s,a)$ si può usare il gradiente per aggiornare i pesi $w$:

- Stima $Q_w(s,a)$
- Target $r(s,a,s') + \gamma \max_{a'} Q_{\overline{w}}(s',a')$, cioè il valore che si otterrebbe se si conoscesse il modello

I pesi si stipano minimizzando una funzione di errore, in questo caso l'errore quadratico:

$$\mathrm{Err}(w) = \left( \underbrace{Q_w(s,a)}_{\text{Stima}} - \underbrace{r(s,a,s') - \gamma \max_{a'} Q_{\overline{w}}(s',a')}_{\text{Valore reale}} \right)^2$$

per farlo si usa il gradiente:

$$\frac{\partial \mathrm{Err}(w)}{\partial w} = 2 \left( Q_w(s,a) - r(s,a,s') - \gamma \max_{a'} Q_{\overline{w}}(s',a') \right) \cdot \frac{\partial Q_w(s,a)}{\partial w}$$

Lo scalare 2 non è rilevante per l'aggiornamento perchè l'unica cosa che conta è la direzione del gradiente.

L'algoritmo del gradient Q-learning è il seguente:

```
Initialize weights w randomly in [-1, 1]
Initialize s {observe current state}
loop
  Select and execute action a
  Observe new state s' receive immediate reward r
  Compute gradient:
    grad = (Q_w(s,a) - r - gamma * max_a' Q_w(s', a')) * partial
    Q_w(s,a) / partial w
  Update weights:
    w = w - alpha * grad
  Update state:
    s = s'
end loop
```

Si nota che non si usa $\overline{w}$ per calcolare il gradiente, ma si usa la stima perchè il valore reale non è noto. Per questo motivo **non c'è garanzia di convergenza**. La convervenza è garantita solo se si usa il **Linear gradient Q-learning**.

## 7.14 Mitigare la divergenza

Ci sono due approcci principali per mitigare la divergenza nel Deep Q-learning:

- Experience replay
- Utilizzare due network diverse:
  - Q-network
  - Target network

### 7.14.1 Experience replay

L'idea è quella di memorizzare in un buffer le esperienze dell'agente (ad esempio $(s, a, s', r)$) e usarle ad ogni step di apprendimento. Ad ogni passo si estrae un batch casuale di esperienze dal buffer e si usa per aggiornare i pesi della rete neurale. I vantaggi sono:

- Riduce la correlazione tra campioni successivi (aumenta la stabilità)
- Riduce il numero di interazioni con l'ambiente (aumenta l'efficienza dei dati)

### 7.14.2 Target network

L'idea è quella di mantenere una rete **target** separata da aggiornare periodicamente, ma non ad ogni step.

- Il Q-network calcola: $Q_w(s, a)$
- Il Target network calcola: $Q_{\overline{w}}(s', a')$

### 7.14.3 Deep Q Network

Il Deep Q Network (DQN) è un algoritmo di Deep Reinforcement Learning che utilizza una rete neurale profonda per approssimare la Q-function $Q(s, a)$. Si utilizzano sia l'experience replay che il target network per migliorare la stabilità e l'efficienza dell'apprendimento. Lo pseudocodice del DQN è il seguente:

```
Initialize weights w and w' randomly in [-1, 1]
Initialize s {observe current state}
loop
  Select and execute action a
  Observe new state s' receive immediate reward r
  Add (s, a, s', r ) to experience buffer
  Sample mini-batch MB of experiences from buffer
  for (s_hat, a_hat, s', r_hat) in MiniBatch do
    grad = (Q_w(s_hat, a_hat) - r_hat - gamma * max_a_hat'(Q_w_hat(
    s_hat', a_hat'))) * partial Q_w(s_hat, a_hat)/partial w
    update weights w <- w - alpha * grad
  end for
  update state s <- s'
  every c steps, update target: w <- w
end loop
```